الوحدة 27

البرمجة العامة

مقدمة

في هذه الوحدة، ستتعلم كيفية تنفيذ أنشطة البرمجة في المؤسسات الهندسية؛ كما تتعرف على مراحل عملية تصميم البرمجيات وتطويرها؛ إلى جانب اختبار البرامج وتسليمها.

كما تُصمِّم برامج لتلبية متطلبات المستخدم وتطورها مع إنتاج وثائق لمرافقة برنامجك. بالإضافة إلى ذلك، ستصمم البرنامج وتطوره باستخدام لغة برمجة مناسبة مع اختبار الحل الخاص بك وتحسينه.

يُعد فهم كيفية عمل برامج الكمبيوتر من المعارف الأساسية للمهندسين الذين يتطلعون إلى العمل في تطوير البرمجيات، مما يعني أن هذه الوحدة ستساعدك على الاستعداد للحياة العملية أو لمزيد من الدراسات.

أهداف التعلم

في هذه الوحدة، ستتمكن من التالي:

(أ) دراسة الأدوات والتقنيات المستخدمة في تطوير برنامج

(ب) تصميم برامج لتلبية متطلبات المستخدم

(ج) تطوير برامج لتلبية متطلبات المستخدم

كيف سيتم تقييمك

يتم تقييم هذه الوحدة داخليًا من خلال موجزات الواجبات الفردية أو المتعددة التي يقدمها لك معلمك. وسيكون عليك تقديم أدلة لإثبات أنك حققت أهداف التعلم. ويمكن تقديم مجموعة الأدلة الخاصة بك في العديد من التنسيقات، سواء إلكترونيًا أو ورقيًا. يسرد مخطط الدرجات في المواصفات لهذه الوحدة ما يجب عليك القيام به للحصول على درجات النجاح والتفوق والامتياز. ستوجهك أنشطة التقييم في هذه الوحدة خلال المهام التي ستساعدك على تحقيق النجاح.

سيخبرك معلمك بالشكل الذي ستتخذه تقييماتك بالضبط، ولكن قد يُطلب منك إعداد عرض تقديمي حول أهمية أدوات وتقنيات البرمجيات في البرمجة، مدعومًا بلقطات شاشة للتعليمات البرمجيات (الأكواد) والبرامج المطورة و/أو وثائق التصميم المكتوبة بما في ذلك كيفية تلبية متطلبات المستخدم والعناصر المحددة للتطوير المستقبلي. قد يُطلب منك أيضًا تخطيط برنامج وإنشائه وتقييمه، مع تقديم دليل على البرمجة واختبار المطور.



الشكل 27.1 مطور برامج يعمل في المكتب في وقت متأخر من الليل.

هدف التعلم (أ): دراسة الأدوات والتقنيات المستخدمة في تطوير برنامج

بدء النشاط

من خلال العمل في مجموعة صغيرة، ناقش ثم أعد قائمة بلغات البرمجة التي ربما سمعت عنها أو استخدمتها من قبل. على سبيل المثال، يتم استخدام HTML لإنشاء مواقع الويب.

لبدء كتابة البرامج، يتعين علينا أولاً فهم الأدوات والتقنيات المتاحة لنا.

تستخدم جميع لغات البرمجة نفس التركيبات لبناء الكتل البرمجية. في هذا الموضوع، سوف نبحث في أنواع أو نماذج لغات البرمجة، وكيفية تمثيلها بلغات البرمجة المختلفة. كما سنتناول أيضًا نماذج التعليمات البرمجية لبدء إنشاء كتل صغيرة من التعليمات البرمجية باستخدام التقنيات القياسية المتوفرة في الكثير من لغات البرمجة الشائعة. في نهاية الموضوع، سنلقي نظرة على أدوات البرمجيات المتاحة لنا لتطوير البرنامج وإصلاحه وتشغيله.

المصطلحات الرئيسية

النموذج: طريقة لتصنيف الكائنات بناءً على الميزات المشتركة.

C#: لغة في إطار مايكروسوفت للغات البرمجة. وهي تستند إلى لغة C، وهي لغة إجرائية، مع ميزات موجهة نحو الكائنات.

(أ1) نماذج لغة البرمجة

قبل أن نبدأ في إنشاء تطبيق باستخدام لغة برمجة، يجب أن نختار اللغة التي تناسب احتياجاتنا. تستخدم أجهزة الكمبيوتر اللغات للتواصل، تمامًا كما نفعل مع لغاتنا الخاصة. تقع جميع لغات البرمجة في "النوع" أو "الفئة" المعروفة بالنموذج. يستخدم هذا المصطلح لتصنيف ميزات لغة البرمجة في مجموعة. تم تصميم بعض لغات البرمجة لدعم نموذج واحد، مثل Smalltalk، بينما تم تصميم بعض لغات البرمجة لدعم العديد من النماذج المختلفة، مثل ®Java أو Visual Basic أو ®C#. في هذه الوحدة، سنستخدم لغة البرمجة C# لأنها لغة متعددة النماذج.

ستتعلم في هذه الوحدة تصميم وتطوير التعليمات البرمجية للبرنامج بإحدى لغاتNET. من مايكروسوفت. تدعم هذه اللغات العديد من النماذج ولكن تركيزنا سيكون على:

- البرمجة الإجرائية
- البرمجة الموجهة نحو الكائنات
- البرمجة القائمة على الأحداث

هل تعلم؟

أصدرت مايكروسوفت لأول مرة إطار NET Framework. الخاص بها في عام 2002. ومنذ ذلك الحين، كان هناك أكثر من 20 إصدارًا مختلفًا من NET. ويُقترح أن ربع مطوري البرامج في جميع أنحاء العالم يستخدمون لغات NET. تم إنشاء اللغة التي سنستخدمها في هذه الوحدة، C#، لأول مرة في عام 2000. وتم تسميتها في الأصل COOL، بمعنى لغة C-live الموجهة نحو الكائنات، ولكن تم تخفيض الاسم إلى C# بسبب قوانين العلامات التجارية.

البرمجة الإجرائية

تتبع لغات البرمجة الإجرائية مجموعة من التعليمات التي تحدد تدفقها. كما أنها تحدد بداية ونهاية للبرنامج. فهي عبارة عن سلسلة من الخطوات التي تكمل المهمة. نحن نقوم بمهام إجرائية في أنشطتنا اليومية.

عندما نقوم بإعداد كوب من الشاي، فإننا نتبع مجموعة نموذجية من التعليمات:

الخطوة 1: أضف الشاي والماء والسكر إلى مقلاة أو غلاية.

الخطوة 2: ضع الخليط على الحرارة لغليه لعدة دقائق.

الخطوة 3: خفف الحرارة واترك الخليط يغلي ببطء.

الخطوة 4: أضف ورقة نعناع إلى الخليط.

الخطوة 5: قدم خليط الشاي في كأس أو كوب شاي.

الخطوة 6: استمتع بتناول الشاي!

يمكن أن يشكل هذا الإجراء البسيط المكون من ست خطوات أساسًا لبرنامج صنع الشاي.

النشاط

في مجموعة صغيرة، فكر في نشاط تقوم به كل يوم يحتوي على مجموعة نموذجية من الخطوات. اكتب كل خطوة تقوم بها في هذا النشاط وقم بإنشاء مجموعة إجرائية من التعليمات التي يمكن استخدامها لإنشاء برنامج لتكرار هذا النشاط. على سبيل المثال، فكر في كيفية استيقاظك ومغادرة منزلك كل يوم أو كيفية طهي وجبة للعائلة والأصدقاء.

البرمجة الموجهة نحو الكائنات

في البرمجة الموجهة نحو الكائنات، نقوم بإنشاء كائنات أو "أشياء". تم تصميم كل منها في صورة "مخططات" تحتوي على **المعلمات** التي تصف الكائن **والإجراءات** التي تسمح له بالتفاعل مع نفسه أو مع الكائنات الأخرى. وفي كل مرة نريد فيها استخدام الكائن الخاص بنا، نقوم بإنشاء **مثيل** أو "إصدار" منه، وتعيين خصائصنا الخاصة وجعله ينفذ الإجراءات. يتم تعريف كل كائن نقوم بإنشائه ضمن مخطط يعرف باسم الفئة. مصطلح "**الفئة**" هو كلمة أساسية في البرمجة الموجهة نحو الكائنات، حيث إن البرنامج عبارة عن مجموعة من فئات عديدة.

في البرمجة يمكننا إنشاء كائن يسمى "سيارات باهظة الثمن". سيحتوي الكائن الخاص بنا على المعلمات: الماركة والطراز واللون. وسوف يحتوي على إجراء يسمى Drive (القيادة).

الجدول 27.1

سيارات باهظة الثمن
المعلمات
الماركة: نص/كلمات (تُعرف باسم السلسلة)
الطراز: نص/كلمات (تُعرف باسم السلسلة)
اللون: النص/الكلمات (تُعرف باسم السلسلة)
الإجراءات
Drive (القيادة)

يمكننا إنشاء مثيلات لكائن Expensive Cars (*السيارات باهظة الثمن*) الخاص بنا لتحديد سيارات معينة. يحتاج برنامجك إلى السماح للمستخدم بقيادة سيارة مرسيدس من الفئة A ذات اللون الأزرق. يمكننا إنشاء كائن بهذه المعلمات بناءً على كائن *السيارات باهظة الثمن*.

المصطلحات الرئيسية

المعلمة: سمة أو خاصية أو وصف يعطي تعريفًا لشيء ما. تصف عبارة "عمر ذو شعر بني *وعيون خضراء*" خاصيتين لعمر.

الإجراء: فعل أو طريقة يمكن أن يقوم بها شيء ما. تحدد عبارة "عمر يركب *دراجته*" الفعل الذي يمكن أن يؤديه عمر.

المثيل: نسخة من كائن يمكن أن يكون له قيمه الخاصة في المعلمات ولكن لا يزال يعتمد على الكائن الأصلي الخاص به.

الفئة: تعريف قالب للطرق والمتغيرات في نوع معين من الكائنات.

النشاط

قم بعمل نسخة من الجدول أعلاه ولكن أكمل قائمة المعلمات الخاصة بك للسيارات باهظة الثمن المختلفة التي تعرفها.

الجدول 27.2

مثال على السيارات باهظة الثمن
المعلمات ماركة السيارة = مرسيدس الطراز = فئة A اللون = أزرق
الإجراءات Drive (القيادة)

عندما "نقود" هذا الكائن، فإننا سنقود سيارة مرسيدس من الفئة A ذات اللون الأزرق.

وفي كل نسخة مختلفة من السيارة *باهظة الثمن*، نعيد استخدام "مخطط" الكائن الخاص بنا في كل مرة. يتيح لنا هذا النهج كتابة كود برمجة فعال ويمكن الاحتفاظ به.

البرمجة القائمة على الأحداث

بدون النموذج *القائم على الحدث*، سيحتاج البرنامج إلى التحقق باستمرار أو "استطلاع" حالة شيء ما لمعرفة ما إذا كان هناك شيء ما قد حدث. ففي البرامج القائمة على الأحداث، يتم تحديد تدفق البرنامج من خلال الأحداث. يمكن أن تكون هذه الأحداث:

- نقرات الماوس على شاشة الكمبيوتر
- المفاتيح الموجودة على لوحة المفاتيح
- المستشعرات داخل الآلة
- دقات عدّاد الوقت
- الضغطات على شاشة الهاتف الذكي
- أحداث مخصصة مثل بث رسالة من جزء من البرنامج إلى آخر.

يمكننا استخدام الأحداث لإخبار البرنامج بما يجب فعله عند حدوث شيء ما. قد ينتظر هذا البرنامج حدوث شيء ما أو يؤدي مهمة بنشاط. يخبر الحدث البرنامج بـ"إيقاف ما تفعله، افعل ذلك بدلاً منه". تستخدم العديد من أنواع التطبيقات المختلفة الأحداث لاتخاذ القرارات.

كبشر، نحن نشبه البرمجة القائمة على الأحداث في استخدامها للأحداث. على سبيل المثال، يقوم قريبك بإعداد البقلاوة في المطبخ أثناء وجودك في غرفتك بالطابق العلوي. وهناك العديد من أفراد الأسرة ينتظرون تناول البقلاوة ولا تريد أن تفوتك الفرصة. يمكنك النزول إلى المطبخ في الطابق السفلي بانتظام للتحقق مما إذا كانت جاهزة، لكنك سئمت من التحقق المستمر. في هذه الحالة، ستقوم "باستطلاع" حالة البقلاوة لمعرفة ما إذا كانت جاهزة. هناك احتمال ضئيل للحصول على الوقت المناسب لأن تكون البقلاوة جاهزة، لكن الاحتمال الأكبر هو أنك لا تحصل على أي شيء على الإطلاق!

بدلاً من الصعود والنزول المستمر، يمكنك أن تطلب من قريبك الصعود إلى الطابق العلوي وإعلامك عندما تكون البقلاوة جاهزة للأكل. يضمن لك هذا الحدث أن تكون على علم باللحظة التي تكون فيها البقلاوة جاهزة ولن تفوتك.

يشرح هذا المثال كيف يمكن للبرنامج أن يعمل باستخدام نوع من العمليات والاستراتيجيات المعرفية: لغة تسمى pseudocode "الكود الزائف". Pseudocode (الكود الزائف) هو طريقة لكتابة كيفية عمل البرنامج دون استخدام أي لغة برمجة محددة. فليس الكود الزائف "كودًا حقيقيًا"، ولن "يعمل " أبدًا كبرنامج. إذ يساعدك على فهم ما سيحدث ويساعد على شرحه للمستخدمين الآخرين. سيستخدم هذا الدليل مزيجًا من الكود الزائف وكود البرمجة.

في البرمجة، يمكنك إعداد كائنين للمثال أعلاه:

"أنت" (الكائن 1) لديك حدث يسمى "أخبرني عندما تكون البقلاوة جاهزة". يشترك "القريب" (الكائن 2) في الحدث الخاص بك ويبلغ عنه عندما تكون البقلاوة جاهزة. ستتلقى "أنت" الإشعار وتنفذ إجراءً.

مراجعة ما تعلمته

قم بإعادة النظر في النشاط من مهمة البرمجة الإجرائية حيث تم تحديد مجموعة من الخطوات لمهمة يومية. أعد هيكلة مجموعة الخطوات الخاصة بك لتشمل العناصر الموجهة نحو الكائنات والعناصر القائمة على الأحداث لتصميم برنامج.

(أ2) تقنيات البرمجة

ثمة العديد من التقنيات المستخدمة في لغات البرمجة لمساعدتنا في كتابة برامج تتسم بالكفاءة والفاعلية. كل لغة لها طريقتها الخاصة في استخدامها، ولكننا سنلقي نظرة على أمثلة في C# والكود الزائف لمساعدتنا على فهم كيفية استخدامها.

قبل أن نبدأ في تناول كتابة التعليمات البرمجية (الكود)، نحتاج إلى فهم بنية البرامج. تتكون البرامج من:

- المعلمات والمتغيرات
- المصفوفات والقوائم
- الدوال والإجراءات
- العوامل البرمجية
- المنطق العلائقي
- المنطق البولياني
- النطاق.

يعد التعليق التوضيحي و"التعليقات" عنصرًا مهمًا في الترميز. تحتوي العديد من اللغات على بنيات ومتطلبات محددة للدلالة على التعليقات. ففي العديد من اللغات، يتم الإشارة إلى التعليقات بواسطة // (خطان مائلان للأمام) لوضع علامة على بداية سطر التعليق (الشكل 27.2). لا يتم تجميع هذه الأسطر ولا تنفيذها حيث يتم تفسيرها بواسطة الكمبيوتر على أنها "ليست تعليمات برمجية".

```
//a single line comment
```

الشكل 27.2 تعليق من سطرواحد.

المصطلحات الرئيسية

زائف: شيء مزيف يحاول أن يمر بوصفه الشيء الحقيقي.

المهارات

المهارات المعرفية:

- التفسير
- التعلم الموائم
- حل المشكلات

يمكن كتابة أسطر متعددة من التعليقات في صورة // لكل سطر فردي، أو /* لبداية كتلة من التعليقات، * لكل سطر لاحق و*/ لإكمال كتلة التعليق (الشكل 27.3).

```
//A block of comments
//that can be used to explain
//how a chunk of code near to it
//works in an easy to read and
//concise block without scrolling
//across the page to read the whole
//comment.

/* An alternate way of commenting
* a lot of lines out for multiple
* lines of comments
*/
```

الشكل 27.3 كتلة التعليقات.

يجب أن تتبع المعلمات والمتغيرات والطرق والدوال مجموعة من الإرشادات التي تحدد كيفية تسميتها. تسمى هذه الإرشادات اصطلاحات التسمية. تساعد اصطلاحات التسمية المبرمجين على إنتاج تعليمات برمجية قابلة للقراءة والاحتفاظ بها. إذ تساعد على إنتاج تعليمات برمجية واضحة وموجزة، وتعطي أوصافًا للمعلمات وتسمح بالبحث عن معلمات محددة بالاسم بطريقة أسهل بكثير. لا توجد مجموعة قياسية ومحددة من اصطلاحات التسمية التي يجب على المبرمجين اتباعها. يستخدم كل مبرمج فهمه الخاص لاصطلاحات التسمية. سيعرض لك هذا الدليل الطريقة الأكثر منطقية لاصطلاحات التسمية الخاصة بهذه الوحدة. إذا أحرزت تقدمًا في مزيد من الدراسات والحياة العملية في هذه الصناعة، حينئذٍ يمكنك تغيير وتطوير نهج اصطلاحات التسمية الخاصة بك مع اكتساب الخبرة كمبرمج.

خذ قصاصة برمجية (الشكل 27.4) على سبيل المثال:

```
3    var a = b * c;
```

الشكل 27.4 قصاصة برمجية لعملية ضرب حسابية بسيطة.

ماذا يحسب هذا الخط؟

إذا أعدنا كتابة القصاصة البرمجية باتباع اصطلاحات التسمية (الشكل 27.5)، فماذا يحسب الآن؟

```
6    var area = height * width;
```

الشكل 27. 5 القصاصة البرمجية لعملية الضرب الحسابية باتباع اصطلاحات التسمية.

يمثل استخدام معرفات ذات دلالة أحد إرشادات التسمية الشائعة. يوضح المثال أعلاه عملية حسابية لمساحة ذات ارتفاع مضروبًا في العرض. هذه الأسماء ليست ذات دلالة كافية والطريقة الممكنة لجعلها ذات دلالة أكبر هي استخدام الكلمات المركبة لوصف المتغير.

إذا تمت كتابة برنامج لحساب مساحات الحدائق المختلفة، فإن القصاصة البرمجية أعلاه ستؤدي إلى تعليمات برمجية غير واضحة ويصعب قراءتها. يمكننا حل هذه المسألة عن طريق إعادة تسمية المتغيرات باستخدام المركبات.

لا يُسمح بالمسافات، لذلك هناك ثلاث تقنيات يمكن استخدامها:

المركبات البسيطة - يتم ببساطة ربط الكلمات معًا بدون مسافات (الشكل 27.6).

```
//compound naming
var garden1area = garden1height * garden1width;
```

الشكل 27.6 التسمية المركبة.

مفصولة بالمحددات - يتم ربط الكلمات معًا باستخدام الشرطات السفلية "_" (الشكل 27.7).

```
//delimeter separated
var garden_1_area = garden_1_height * garden_1_width;
```

الشكل 27.7 تسمية مفصولة بالمحدد.

فصل الأحرف - يتم فصل الكلمات بأحرف كبيرة، تُعرف باسم "camel-case" (نسق الجمل) (الشكل 27.8).

```
//letter-case or camelCase
var garden1Area = garden1Height * garden1Width;
```

الشكل 27.8 نسق الأحرف أو نسق الجمل.

هل تعلم؟

تمت تسمية Camel Case (نسق الجمل) على اسم الحدبة الأولية للحرف الكبير داخل الاسم، حيث تشبه سنام الجمل.

المعلمات والمتغيرات

لقد تناولنا هذا باختصار في الموضوع الموجه نحو الكائنات. يتم استخدام المعلمات والمتغيرات لتخزين المعلومات أثناء تنفيذ البرنامج. ويتم تعريفها بالعبارة التالية:

<DataType(نوع البيانات)> <name (اسم)> <<equals a value (تعادل قيمة)>>*

* تعيين قيمة للمعلمة اختياري.

ثمة العديد من أنواع البيانات المختلفة التي يمكن تخزينها في المعلمات أو المتغيرات. يجب أن يكون لكل معلمة أو متغير نوع. هذه هي الأنواع التي سننظر إليها ونستخدمها في هذه الوحدة، موضحة بمزيد من التفصيل.

المصطلحات الرئيسية

نوع البيانات: سمة تصف جزءًا من البيانات وتخبر جهاز الكمبيوتر بكيفية قراءتها وكتابتها.

السلسلة

السلسلة عبارة عن مجموعة متسلسلة من الأحرف التي تشكل كلمة أو جملة. يتم التعامل معها كمصفوفة من الأحرف، مما يعني أنه يمكنك استخراج أحرف مفردة من سلسلة أو معالجة السلسلة ككل. ثمة العديد من الخصائص والعمليات "المضمنة" داخل C# والتي يمكن تنفيذها على السلاسل. يمكنك الإعلان عن سلسلة في التعليمات البرمجية الخاصة بك باستخدام الكلمة

الأساسية *String* أو *string* متبوعة باسمها (الشكل 27.9). C# هي لغة قوية جدًا، ويمكنها "التنبؤ" بنوع البيانات التي تستخدمها بناءً على قيمتها. ستبدأ الأمثلة أدناه في استخدام الكلمة الأساسية "var". لا توجد حدود مفروضة في أداء برنامج يستخدم var بدلاً من نوع البيانات للمتغير الخاص بك. لا يمكنك إنشاء سطر **var** بدون قيمة لأن التحويل البرمجي لا يعرف نوع البيانات إلى أن تتم عملية التعيين. يمكن استخدام الكلمة الأساسية **var** عندما يكون نوع البيانات واضحًا ولا لبس فيه.

```
//The value is Programming Report with spaces at the end
string myReportName = "Programming Report    ";
String myReportName = "Programming Report    ";
```

الشكل 27.9 تعريف المعلمات *كسلاسل*.

يمكنك معالجة السلسلة باستخدام العوامل البرمجية المضمنة (الشكل 27.10).

```
//A string is classed as an array
//would contain the character 'r'
var fifthLetter = myReportName [4];

//Split the string by the space in to smaller chunks,
//this would produce a collection containing Programming, Report and some empty strings.
var splitBySpace = myReportName.Split(' ');

//this would create a string with a value of Programming Document
var replacedWord = myReportName.Replace("Report", "Document");

//this would get the first 7 letters of Programming Report
var firstSevenLetters = myReportName.Substring(0, 7);

//trim spaces off the end
var trimmedString = myReportName.TrimEnd();

//to Uppercase or lowercase
var toUpperCase = myReportName.ToUpper();
var toLowerCase = myReportName.ToLower();
```

الشكل 27.10 - دوال معالجة السلسلة.

المصطلحات الرئيسية

عدد صحيح: العدد الصحيح هو رقم صحيح في البرمجة. ولا يمكن أن يحتوي على كسر أو رقم عشري.

يمكننا استخراج حرف واحد من سلسلة من خلال معالجة السلسلة كمصفوفة. تطلب الأقواس القائمة الزاويتين التي تحتوي على عدد صحيح بداخلها من جهاز الكمبيوتر استخراج القيمة حيث يكون الفهرس n (n هو الرقم الموجود داخل القوس).

يمكننا تقطيع سلسلة إلى مجموعة من السلاسل الأصغر باستخدام دالة **Split (تقسيم)** وإخبارها بما سيتم تقسيم السلسلة به. في المثال (الشكل 27.10) نستخدم ' ' أو 'space' (مسافة) لتقسيم السلسلة بها.

يمكننا استبدال الأحرف أو الكلمات الكاملة أو أجزاء الكلمات باستخدام دالة **Replace (الاستبدال)**.

يمكننا تحويل جزء من سلسلة إلى سلسلة جديدة باستخدام دالة **Substring (السلسلة الفرعية)**. تطلب هذه الدالة فهرس البداية (نحسب من الصفر) والطول.

يمكننا إزالة أي مسافات بادئة أو لاحقة من سلسلة باستخدام دالة **Trim End**. تتوفر دالة **Trim** و**Trim Start**. تنطبق دالة **Trim** على البداية والنهاية، بينما تنطبق دالة **Trim Start** و**Trim End** فقط على بداية السلسلة ونهايتها.

يمكننا تغيير نسق السلسلة عن طريق إجبار جميع الأحرف بداخلها على أن تكون في نسق الأحرف الكبيرة أو الصغيرة باستخدام الدوال **ToUpper** و**ToLower**.

تحتوي معظم أنواع البيانات على دوال مضمنة لتحويل البيانات من نوع بيانات إلى آخر. فعندما تكون قيمة الإدخال عبارة عن String (سلسلة)، قد يحتوي نوع البيانات الهدف على دالة Parse(string). وعندما تكون قيمة الإدخال نوع بيانات آخر ونوع البيانات الهدف عبارة عن سلسلة، تكون الدالة المطلوبة هي **ToString( )**.

وبينما نتنقل عبر أنواع البيانات الأخرى التي تغطيها هذه الوحدة، سنغطي دوال التحويل المتاحة لنا.

أخيرًا، تحتوي السلاسل على خاصية واحدة مفيدة بشكل خاص تخبرنا بطول السلسلة وعدد الأحرف الموجودة فيها (الشكل 27.11).

```
//string properties
int characterCount = myReportName.Length;
```

الشكل 27.11 ابحث عن طول سلسلة.

النشاط

تعد الحلقات التكرارية مفيدة جدًا للتنقل عبر مصفوفات البيانات. وكما هو مذكور أعلاه، السلسلة هي ببساطة مصفوفة من الأحرف. يمكننا استخدام الحلقات التكرارية للتنقل عبر كل حرف في تلك المصفوفة وتنفيذ الإجراءات.

1. اكتب برنامجًا للتنقل بين كل حرف في الكلمة وإخراج حرف واحد في كل سطر إلى وحدة التحكم (**Console.out.WriteLine**).
2. قم بتكييف برنامجك لقبول المدخلات من المستخدم (**Console.in.Readline**) لتطبيق وحدة التحكم)، وقم بتكرار كل حرف وإخراج حرف واحد في كل سطر.

العدد الصحيح

العدد الصحيح، المعروف أيضًا باسم 'int'، هو رقم صحيح بدون منازل عشرية. ففي البرمجة، يشير int عادةً إلى رقم 32 بت ويمكن تمثيله في صورة Int32 كنوع بيانات. يعتمد نطاق القيم التي يمكن تخزينها في Int32 على الدلالات الموقعة أو غير الموقعة. يمكن لـ Int32 غير الموقّع (UInt32) تخزين أي قيمة تتراوح بين 0 و4,294,967,295 (يساوي 2^{32}-1). ويمكن لـ Int32 الموقّع (Int32) تخزين أي قيمة تتراوح بين –2,147,483,648 و2,147,483,647 (يساوي —2^{31} إلى 2^{31}-1). لاحظ أن الرقم 32 الذي يأتي بعد Int يمثل أس 2 لحساب محتوياته. كما يتوفر Int64 أيضًا لتمثيل الأرقام الأكبر ويتم تمثيله إما بـInt64 أو UInt64 أو long أو ulong في C#.

مراجعة ما تعلمته

ما هي القيمة القصوى لـ UInt64؟ ما هي القيم الدنيا والقصوى لـ Int64؟

دعونا نجرب بعض العمليات الحسابية البسيطة باستخدام الأعداد الصحيحة في البرمجة. ما ناتج البرنامج في الشكل 27.12؟

```
1    int sum = 0;
2    sum = 1 + 2;
3    int a = 10;
4    int b = 20;
5    sum = a * b;
6    Console.Out.WriteLine(sum);
```

الشكل 27.12 برنامج حساب عدد صحيح.

كما هو الحال مع السلاسل، هناك العديد من الدوال المضمنة في نوع بيانات العدد الصحيح في C#.

مراجعة ما تعلمته

ما هي قيم كل سؤال في القصاصة البرمجية أدناه؟

```
int testNumber = 101;

var question1 = int.IsEvenInteger(testNumber);
var question2 = int.IsOddInteger(testNumber);
var question3 = int.IsNegative(testNumber);
var question4 = int.IsPositive(testNumber);
var question5 = int.Parse("100");
```

الشكل 27.13 برنامج حساب عدد صحيح باستخدام دوال "Is".

المصطلحات الرئيسية

القيمة المنطقية: قيمة صواب أو خطأ، يمكن اختصارها إلى "bool".

القيمة المنطقية

إذا أجبت بـ"نعم" أو "لا" على الأسئلة 1 و2 و3 و4 في مراجعة ما تعلمته أعلاه، فأنت على صواب تقريبًا. تقوم دوال "Is" أعلاه بإخراج قيمة منطقية. القيمة المنطقية أما أن تكون TRUE (صواب) أو FALSE (خطأ). تعد أنواع البيانات المنطقية مفتاحًا للقرارات المتخذة في برنامج الكمبيوتر. ويمكن حساب جميع القرارات تقريبًا على أنها true (صواب) أو false (خطأ).

دعونا نحدد بعض المعلمات المنطقية في التعليمات البرمجية (الشكل 27.14).

```
1    bool isThisValueTrue = true;
2    bool isThisValueFalse = false;
3
4    //what is the value of each of the boolean parameters above?
5
6    var trueString = bool.TrueString;
7    var falseString = bool.FalseString;
8
9    var parsedBoolean = bool.Parse("true");
```

الشكل 27.14 تحديد المعلمات المنطقية.

لنلقِ نظرة على السطرين 1 و2. يعرّف السطر 1 معلمة تسمى **'IsThisValueTrue'** ويقوم بتعيين كلمة "*صواب*" لها. فإذا تم استخدام هذه المعلمة في القرار، فسيتم حلها على أنها TRUE (صواب).

يحدد السطر 2 معلمة تسمى **"IsThisValueFalse"** ويقوم بتعيين خطأ لها. فإذا تم استخدام هذه المعلمة في القرار، فسيتم حلها على أنها FALSE (خطأ). لا يحدد اسم المعلمة القيمة المتضمنة بداخلها.

سينتج السطر 6 و7 سلاسل للقيم المنطقية الصواب والخطأ.

سيقوم السطر 9 بتحويل أو "توزيع" قيمة سلسلة إلى قيمة منطقية.

مضاعف

بالنسبة للأعداد الصحيحة، فإننا نستخدم الأعداد الصحيحة، ولكن لا يمكن لكل برنامج التعامل مع الأرقام الصحيحة فقط. إنه يحد بشدة من الدقة عند إجراء حتى أبسط العمليات الحسابية. لمزيد من الدقة يمكننا استخدام **Double** (**المضاعف**). المضاعف هو نوع بيانات يخزن أرقام عشرية عالية الدقة. يمثل المضاعف "الوسط" لأنواع البيانات ذات الأرقام العشرية ويعطي

حوالي 15 إلى 17 رقمًا من الدقة. وللحصول على دقة أقل (من 6 إلى 9 أرقام)، يتوفر **نوع بيانات الأرقام العشرية**، ولمزيد من الدقة (28 إلى 29 رقمًا)، يتوفر **نوع بيانات الكسور العشرية**. تتشابه التعليمات البرمجية للبيانات المضاعفة والأرقام العشرية والكسور العشرية بشكل كبير جداً ويمكن تحويلها فيما بينها. لكن احذر من أن التحويلات بين الأرقام العشرية يمكن أن تؤدي إلى فقدان الدقة وهو أمر أساسي في العديد من تطبيقات الأعمال والهندسة.

يتمثل الاستخدام الأكثر شيوعًا لـ **Double (المضاعف)** داخل تطبيق الأعمال أو الهندسة في حساب **هامش الربح**.

هامش الربح (أو **هامش الربح الإجمالي**) هو حساب **(قيمة المبيعات - قيمة التكلفة) /قيمة المبيعات) * 100**.

لنفترض أن **قيمة المبيعات** لدينا تساوي 1000 بالضبط، **وقيمة التكلفة** لدينا تساوي 600.44. ما هو **هامش الربح الإجمالي** لدينا؟

يعطي الحساب (1000 — 600.44)/1000 ناتج 0.39956، ثم يتم ضربه في 100 لنحصل على 39.956%.

لنقم بإنشاء برنامج لتشغيل هذا العملية الحسابية (الشكل 27.15)

```
public partial class Program
{
        0 references
        private static void Main(string[] args)
        {
                static void Main()
                {
                        double salesValue = 1000;
                        double costValue = 600.44;
                        double gpm = (salesValue - costValue) / salesValue * 100;
                        Console.Out.WriteLine(gpm);
                }
        }
}
```

الشكل 27.15 برنامج حساب هامش الربح الإجمالي.

في الشكل 27.15، حددنا متغيراتنا على أنها **مضاعفة**. قد يؤدي استخدام الكلمة الأساسية **var** لتطبيق نوع البيانات تلقائيًا إلى قيم غير مضاعفة بشكل صريح، يتم تعيينها لأنواع بيانات أخرى.

```
readonly struct System.Int32
var salesValue = 1000;
```

الشكل 27.16 تعيين مضاعف غير صحيح.

يؤدي التمرير فوق الكلمة الأساسية "var" في Visual Studio® IDE (المزيد عن IDEs "بيئات التطوير المتكاملة" لاحقًا في هذا الدليل) إلى إظهار نوع البيانات المحدد للمتغير (الشكل 27.16). قارن ذلك بتعريف المضاعف (الشكل 27.17).

```
readonly struct System.Double
double dblSalesValue = 1000;
```

الشكل 27.17 تعيين المضاعف الصحيح.

نموذج DateTime

يتم تسمية نموذج **DateTime** بشكل مناسب باسم "كائن التاريخ والوقت". يمكن أن يشير هذا الكائن إلى الآن أو أمس أو غدًا أو قبل ساعة أو قبل عام أو حتى مائة عام في المستقبل. تتمثل الميزة الأكثر فائدة في **DateTime** في الحصول على التاريخ والوقت بالضبط الآن. نموذج **DateTime** هو نوع بيانات قوي في لغات البرمجة.

```
var now = DateTime.Now;
var tomorrow = DateTime.Now.AddDays(1);
var yesterday = DateTime.Now.AddDays(-1);
var dateString = "2030-01-01"; //string representation for Year-Month-Day for 1st Jan 2030
var date = DateTime.Parse(dateString);
var nowString = now.ToString("yyyy-MM-dd");
```

الشكل 27.18 تعيينات *التاريخ والوقت*.

في القصاصة البرمجية الموضحة في الشكل 27.18، يمكننا استرداد التاريخ والوقت الآن. كما يمكننا أيضًا معالجة التاريخ والوقت في اليوم السابق أو اليوم التالي، ويمكن أن ينطبق ذلك أيضًا على أجزاء السنة والشهر والساعة والدقيقة والثانية والملي ثانية. يمكن "توزيع" التواريخ من سلسلة إلى **DateTime**، وتحويلها من **DateTime** إلى تنسيق سلسلة قابل للقراءة. إذا كان تنسيق التاريخ في السلسلة معروفًا، حينئذٍ يمكن تحويله إلى **DateTime**، على سبيل المثال، التاريخ ليس دائمًا 01-01-2030 (السنة - الشهر-اليوم)، أحيانًا يمكن أن يكون اليوم/الشهر/السنة أو الشهر/اليوم/السنة. توضح لنا القصاصة البرمجية في الشكل 27.19 كيف يمكن توزيع تنسيقات التاريخ المختلفة.

```
var dateString1 = "28/12/1960";
var dateString2 = "12/28/2031";
var dateString3 = "13.02.2023";

var date1 = DateTime.ParseExact(dateString1, "dd/MM/yyyy", CultureInfo.InvariantCulture);
var date2 = DateTime.ParseExact(dateString2, "MM/dd/yyyy", CultureInfo.InvariantCulture);
var date3 = DateTime.ParseExact(dateString3, "dd.MM.yyyy", CultureInfo.InvariantCulture);
```

الشكل 27.19 توزيع تنسيقات التاريخ المختلفة.

المصفوفات والقوائم

حتى الآن، قمنا بالتحقيق في مثيلات أحادية من البيانات داخل البرمجة. ولاستخدام العديد من العناصر من نفس النوع، يمكننا استخدام *المصفوفات والقوائم* لتخزين المعلومات. يتم استخدام كل من المصفوفات والقوائم لتخزين المعلومات، ويمكن تكرارها، ويمكن فهرستها، كما أنها قابلة للتغيير. لكن هناك بعض الاختلافات الرئيسية بين الاثنين.

المصطلحات الرئيسية

قابل للتغيير: شيء قابل للتغيير.

المصفوفات

المصفوفات هي بنية بيانات أصلية. وتستند إلى بنية الفهرسة. وتتمثل ميزاتها الرئيسية في الآتي:

- لها بنية وطول ثابتان
- تشغل قدرًا معينًا من الذاكرة
- تستند إلى الفهرسة مع وجود العنصر الأول في أدنى الفهرس والعنصر الأخير في الأعلى
- تتميز بالكفاءة في استخدام الذاكرة.

ومع ذلك، فإن الجوانب السلبية لاستخدام المصفوفات تشمل حقيقة أنه:

- يمكن أن يكون تغيير الحجم صعبًا وغير فعال
- لا يمكن إدراج العناصر وحذفها بسهولة
- يتطلب ترتيب المصفوفة وفرزها مصفوفة جديدة.

ويمكننا استخدام مصفوفة لتخزين درجات فصل من الطلاب. إذا حصل المعلم على خمس درجات للطلاب من أصل 100 بترتيب تصاعدي:

25، 33، 44، 45، 67

يريد المعلم معرفة أصغر درجة وأعلى درجة.

من مجموعة الأرقام الصغيرة هذه يمكننا أن نرى أن أصغر درجة هي *23* وأعلى درجة هي *67*. هذا أمر بسيط مع 5 درجات فقط، ولكن ماذا لو كانت قائمة الدرجات هي 20. يمكن للمعلم كتابة برنامج (الشكل 27.20) لتخزين هذه الدرجات العشرين في صورة مصفوفة من الأعداد الصحيحة وكتابة برنامج للحصول على أدنى درجة وأعلى درجة.

```
1    int[] grades = new int[20] {10,12,25,34,36,37,38,45,46,52,55,55,56,57,58,61,62,63,66,68};
2
3    int smallest = grades [0];
4    int largest  = grades [19];
5
6    Console.Out.WriteLine("The smallest is " + smallest);
7    Console.Out.WriteLine("The largest is " + largest);
8
```

الشكل 27.20 درجات الطالب في مصفوفة.

دعونا نلقي نظرة على هذا البرنامج سطرًا بسطر.

يقوم *السطر 1* بتهيئة المصفوفة. نحن نحدد نوع البيانات وحجمها ومحتوياتها داخل هذا السطر. يعرّف '**Int [ ]**' نوع البيانات كمصفوفة من الأعداد الصحيحة. لا تحتوي المثيلات الأولى لـ **int [ ]** على حجم يتم إضافته بعد الكلمة الأساسية "جديد" لتحديد حجم 20. هذا يخبر الكمبيوتر أن هذه المصفوفة ستحتوي على 20 عددًا صحيحًا، وليس أكثر من ذلك. بعد تعريف نوع البيانات النهائي، نقوم بملء محتويات المصفوفة بين الأقواس المعقوفة.

يحصل *السطر 3* على القيمة الموجودة في الفهرس 0 (نحسب من الصفر في البرمجة) ويعينها على أصغر معلمة.

يحصل *السطر 4* على القيمة الموجودة في الفهرس 19 ويقوم بتعيينها إلى أكبر معلمة.

يمكن للمصفوفات تخزين العديد من أنواع البيانات المختلفة، ولكن يجب أن تكون من نفس النوع. ستنتج الكثير من الدوال الأساسية في C# مصفوفات من البيانات لمزيد من المعالجة. ستنتج الدالة التي قمنا بدراستها سابقًا مصفوفة من السلاسل من سلسلة أصلية واحدة. هل يمكنك معرفة الدالة السابقة في السلاسل التي قد تنتج مصفوفة؟

القوائم

يتم إنشاء القوائم فوق المصفوفات ولكنها تنطوي على مزايا وعيوب. وكما هو الحال مع المصفوفات، فإنها تحتوي على مجموعات من البيانات، ولكن محتوياتها لا تقتصر على نوع بيانات واحد محدد. كما أنها مفتوحة لدوال المعالجة المضمنة في إطار عمل .NET. تتمثل ميزاتها الرئيسية في الآتي:

- يمكن أن تختلف أطوالها، فهي "ديناميكية"
- يمكن إدراج البيانات وإزالتها بشكل متكرر
- بنية غير قائمة على الفهرس
- أصبح الترتيب والفرز والمعالجة أسهل.

ومع ذلك، تنطوي على جوانب سلبية هي على النحو التالي:

- تشغل مساحة ذاكرة أكبر من المصفوفة
- لا تتوافق دائمًا مع اللغات الأخرى.

دعونا نلقي نظرة على درجات معلمنا مرة أخرى. حصل المعلم على 20 درجة جديدة ليست بترتيب معين.

```
1    List<int> grades = new List<int>{38,45,46,61,62,63,66,52,58,68,55,10,12,25,34,36,37,55,56,57};
2
3    var sortedGrades = grades.Sort();
4    var smallest = grades.Min();
5    var largest = grades.Max();
6
7
8    Console.Out.WriteLine("The smallest is " + smallest);
9    Console.Out.WriteLine("The largest is " + largest);
```

الشكل 27.21 برنامج درجات الطلاب المرتبة.

دعونا نلقي نظرة على برنامجنا المحدث في الشكل 27.21.

يقوم *السطر 1* بتهيئة قائمة درجات بالأعداد الصحيحة الخاصة بنا. وجرت العادة على أننا نحدد نوع البيانات واسم المعلمة في الجزء الأول من السطر. ثم نقوم بتعيين البيانات لها بعد علامة التساوي باستخدام الكلمة الأساسية "جديد" ونوع البيانات ومحتويات البيانات.

يقوم السطر 3 "بترتيب" المصفوفة الخاصة بنا ترتيبًا تصاعديًا. فإذا كنت تريد ترتيب البيانات ترتيبًا تنازليًا، فاستخدم **Reverse( )** بدلاً من **Sort( )**.

يأخذ *السطر 4* القيمة الأدنى من القائمة.

ويأخذ *السطر 5* القيمة القصوى من القائمة.

مراجعة ما تعلمته ✓

ألق نظرة على بعض الدوال الأخرى المتاحة **للقوائم** في C#. ما الذي يمكنك القيام به أيضًا في **القوائم**؟

الدوال والإجراءات

يتم استخدام الدوال والإجراءات والطرق والروتين الأساسي والروتين الفرعي بالتبادل. في البرمجة العامة، لا يوجد فرق حقيقي بينهما، ويتم استخدامها جميعًا لوصف برنامج فرعي داخل برنامج أكبر. يقتصر مصطلح "الطريقة" في الغالب على اللغات الموجهة نحو الكائنات. لا تقبل جميع أسماء الروتين الفرعي هذه أيًا أو جزءًا واحدًا أو أكثر من البيانات وقد تقوم أو لا تقوم بإخراج البيانات. ولتعريف الدوال والإجراءات بشكل عام خارج لغات البرمجة الموجهة نحو الكائنات، تقوم *الدالة* بإرجاع قيمة، بينما لا يقوم *الإجراء* بذلك.

سنقوم بكتابة *دالة وإجراء* في التعليمات البرمجية. ستقوم الدالة بإرجاع قيمة، علمًا بأن الإجراء لن يقوم بذلك. نحن نبني كشك فواكه مع:

- فواكه مخزنة
- القدرة على إضافة مخزون إلى كشكنا
- طريقة لإعطاء العميل الفاكهة
- طريقة لإظهار أنواع الفاكهة للعملاء المحتملين.

سيشتري العملاء الفاكهة بناءً على الألوان التي يفضلونها. سنقوم ببناء كشك الفاكهة الخاص بنا مع كائن لتخزين أسماء الفاكهة والألوان فيه. سيطلق على هذا اسم **StockedFruit** (الفاكهة المخزنة) (الشكل 27.22).

```
1    public class StockedFruit
2    {
         2 references
3        public string fruitName;
         2 references
4        public string fruitColour;
5    }
```

الشكل 27.22 فئة *StockedFruit* (الفاكهة المخزنة).

تضيف *إجراءات الفاكهة* لدينا مخزونًا إلى كشك الفاكهة الخاص بنا.

```
33       //Procedure - when fruit needs stocking up
         3 references
34       public void StockUpOnFruit(string fruitName, string fruitColour)
35       {
36           fruitsInStock.Add(new StockedFruit()
37           {
38               fruitColour = fruitColour,
39               fruitName = fruitName
40           });
41       }
```

الشكل 27.23 طريقة لتخزين الفاكهة.

تحصل *دالة الفاكهة* لدينا على اسم الفاكهة بناءً على لونها (الشكل 27.24).

```
12   //Function - returns a Fruit string based on colour input
     0 references
13   public string GetFruit(string colourOfFruit)
14   {
15       foreach(var fruitInStock in fruitsInStock)
16       {
17           if (fruitInStock.fruit Colour == colourOfFruit)
18           {
19               return fruitInStock.fruitName;
20           }
21       }
22       return "Sorry, we don't have that fruit";
23   }
```

الشكل 27.24 *دالة* الحصول على الفاكهة بناءً على لونها.

لدينا برنامج فرعي آخر في برنامجنا يسمى **ShowFruit** (الشكل 27.25). وبناءً على بنيته وإجراءاته، هل هو إجراء أم دالة؟

```
37   //Showed when we want to display a specific fruit
     0 references
38   public void ShowFruit(StockedFruit fruit)
39   {
40       Console.Out.WriteLine("Here is our brilliant " + fruit.fruitColour + " " + fruit.fruitName);
41   }
```

الشكل 27.25 البرنامج الفرعي لعرض الفاكهة.

هذا صحيح! إنه إجراء. لا يحتوي على نوع إرجاع.

كما ذكرنا في بداية هذا القسم، لا يوجد فرق كبير بين *الدوال والإجراءات*. يمكن الإشارة إليهما كـ*طرق* في البرمجة الموجهة نحو الكائنات. قد تبدأ التعليمات البرمجية الخاصة بك *بإجراء* ما، ولكنها تصبح بعد ذلك دالة أثناء قيامك بمعالجتها وإعادة بنائها وفقًا لاحتياجاتك وأيضًا من أجل كفاءة البرنامج. إذا أشرت إلى *دالة كطريقة* أو *إجراء* كروتين فرعي أو *طريقة كإجراء*، فسيفهم ذلك مبرمج آخر ما تقصده. يمكن العثور على التعليمية البرمجية الكاملة لبرنامج كشك الفاكهة في ملحق هذا الدليل.

أنواع الإرجاع

تحدد أنواع الإرجاع نوع بيانات المعلومات التي يتم إرجاعها من دالة. وكما هو الحال مع المعلمات والمتغيرات والمدخلات في الطرق أو الدوال أو الإجراءات، تتطلب الدوال تعريف ما يُتوقع إرجاعه. يتم عرض مجموعة متنوعة من أنواع الإرجاع في الشكل 27.26.

```
//input values are whole numbers as a string
int ConvertStringToInteger(string number)
{
    return int.Parse(number);
}
//input values are either true or false as string
bool CheckIsTrueOrFalse(string answer)
{
    return bool.Parse(answer);
}
//input values are two whole numbers as integers
double DivideTwoNumbers(int numberOne, int numberTwo)
{
    return numberOne / numberTwo;
}
//splits a string by its space
string[] SplitSentenceBySpaces(string sentence)
{
    return sentence.Split(' ');
}
//combines three strings and returns a list of strings
List<string> SplitSenticeBySpacesToList(string fruitone, string fruitTwo, string fruitThree)
{
    return new List<string>() { fruitone, fruitTwo, fruitThree };
}
```

الشكل 27.26 دوال لإرجاع الأعداد الصحيحة والقيم المنطقية والمضاعفات والمصفوفات وقو انم البيانات.

مراجعة ما تعلمته ✓

اكتب دالة تأخذ عددًا صحيحًا كمدخل، وتحوله إلى سلسلة (نص) وتُرجع تلك السلسلة.

الاستدعاء الذاتي (المعاودة)

الاستدعاء الذاتي هو استخدام كتل التعليمات البرمجية المتكررة لحل المسائل. يمكن للروتين الفرعي التكراري أن يستدعي نفسه من داخل نفسه. يمكن تشبيه الاستدعاء الذاتي في صورة طريقة نمو الشجرة. توجد شجرة صنوبر في قطعة أرض مفتوحة. تنمو أكواز الصنوبر على الشجرة ثم تتساقط على الأرض. وعندما تصبح أكواز الصنوبر جاهزة، تطلق البذور لبدء نمو شجرة جديدة. ومن ثم يصبح لديك شجرة صنوبر جديدة، وتتكرر العملية.

دعونا نكتب بعض التعليمات البرمجية لبرنامج زراعة الأشجار المتكرر (الشكل 27.27).

```
1    PineTree GrowANewPineTree(Pinecone cone)
2    {
3        return GrowANewPine Tree(cone);
4    }
```

الشكل 27.27 برنامج زراعة الأشجار المتكرر.

هذه معاودة (استدعاء ذاتي) بسيطة لزراعة أشجار الصنوبر. على الرغم من أن هذا ليس برنامجًا مجديًا، إلا أنه سيستمر في زراعة أشجار الصنوبر ولن يتوقف أبدًا.

العوامل البرمجية

العوامل في لغات البرمجة هي رموز تخبر جهاز الكمبيوتر بإجراء عمليات رياضية أو علائقية أو منطقية محددة والتوصل إلى نتيجة.

العوامل الحسابية أو الرياضية

تستخدم البرامج على نطاق واسع للعوامل الحسابية. فيما يلي العوامل الحسابية الشائعة المستخدمة في لغات البرمجة:

+ الجمع، جمع القيم معًا

- الطرح، طرح القيم من بعضها البعض

* الضرب، ضرب القيم معًا

/القسمة، تقسم الأرقام على بعضها البعض

يعطي Mod، أو **باقي القسمة**، ما تبقى من قسمة عدد صحيح (رائع لمعرفة ما إذا كان الرقم قابلاً للقسمة على رقم آخر بالتساوي!) على سبيل المثال عند استعمال الدالة mod في قسمة 6 على 3 يكون الباقي هو 0، وهو قابل للقسمة بالتساوي. عند استعمال الدالة mod في قسمة 8 على 3 يكون الباقي هو 2، وهو غير قابل للقسمة بالتساوي.

المصطلحات الرئيسية

باقي القسمة: ينتج قيمة متبقية من قسمة رقمين.

```
1    int add = 23434 + 27868;
2    int sub = 345345345 – 1523422;
3    int mul = 345 * 45;
4    int div = 344534 / 453;
5    int mod = 550 % 5;
```

الشكل 27.28 الحسابات الرياضية.

في الشكل 27.28 توجد بعض العمليات الحسابية باستخدام كل نوع من أنواع العوامل الحسابية. كم عدد العمليات التي يمكنك التدرب عليها؟ لاحظ أن استخدام الآلة الحاسبة هو استخدام برنامج لذلك لا داعي للغش!

يمكن لبرنامج كمبيوتر إجراء هذه الحسابات البسيطة في جزء من الثانية.

المنطق العلائقي

يساعد المنطق العلائقي على مقارنة قيم البيانات. عوامل المنطق العلائقي الشائعة هي:: == أو تساوي، تتحقق مما إذا كانت قيمتان لهما نفس القيمة.

!= أو لا تساوي، يتحقق مما إذا كانت القيمتان ليس لهما نفس القيمة أو أكبر من، يتحقق مما إذا كانت إحدى القيم أكبر من الأخرى، وما إذا كانت القيمة على يسار العامل البرمجي أكبر من القيمة الموجودة على اليمين.

< أو أقل من، يتحقق مما إذا كانت إحدى القيم أقل من الأخرى، وما إذا كانت القيمة الموجودة على يسار العامل البرمجي أقل من القيمة الموجودة على اليمين.

>= أو أكبر من أو يساوي هو ضم > و =

<= أقل من أو يساوي ل، هو ضم < و =

مراجعة ما تعلمته ✓

```
1    int a = 100;
2    int b = 30;
3
4    bool equalTo = a == b;
5    bool notEqualTo = a != b;
6    bool gt = a > b;
7    bool lt = a < b;
8    bool gte = a >= b;
9    bool lte = a <= b;
```

الشكل 27.29 المنطق العلائقي.

تظهر التعليمات البرمجية الموجودة في الشكل 27.29 عوامل المنطق العلائقي المستخدمة. ما الإجابات على كل سطر من الأسطر 4 إلى 9؟

المنطق البولياني

المنطق البولياني عبارة عن عوامل برمجية على قيم True (صواب) وFalse (خطأ) داخل البرمجة. وهي

- AND (و) - إذا كان كلا العاملين صوابًا، فإن نتيجة هذا العامل تكون TRUE (صواب)
- OR (أو) - إذا كان أي من العاملين صوابًا، فإن نتيجة هذا العامل تكون TRUE (صواب)
- NOT (لا)- إذا كان العامل true (صواب)، فإن نتيجة هذا العامل هي FALSE (خطأ) (فهو يعكس TRUE إلى FALSE، وFALSE إلى TRUE)
- XOR - إذا كان كلا العاملين TRUE (صواب)، فإن النتيجة تكون FALSE (خطأ)

```
int a = 30;
int b = 30;
int c = 23;
int d = 24;

bool andLogic = a == b && c < d; //TRUE
bool orLogic = a > b || c < d; //TRUE
bool notLogic = !(a == b); //FALSE
bool xorLogic = andLogic != notLogic; //!= is the equivalent of XOR in C#, this would equal TRUE
```

الشكل 27.30 المنطق البولياني.

النطاق

النطاق هو ما يحدد رؤية المعلمات والطرق في البرنامج. يمكن اعتبار نطاق البرامج بمثابة أمان البيانات داخل كل منطقة من البرنامج. إذ يوجد ضمن برمجةNET. ، ثلاثة دلالات للنطاق: العام والخاص والمحمي.

النطاق العام يجعل هذه المعلمة أو الطريقة يمكن الوصول إليها من أي مكان، ومن برامج أخرى، ومثيلات أخرى داخل الكائن.

النطاق الخاص يجعل هذه المعلمة أو الطريقة يمكن الوصول إليها فقط من فئتها الخاصة.

النطاق المحمي يجعل هذه المعلمة أو الطريقة مرئية في جميع الفئات المشتقة أو الممتدة منها، بما في ذلك، الفئة الأصلية.

يتم استخدام المصطلحات "العام والخاص والمحمي" داخل التعليمات البرمجية لتحديد النطاق، حتى يعرف الكمبيوتر بعد ذلك تفسير نطاق الطريقة والتحكم فيه. تقوم معظم بيئات التطوير المتكاملة الحديثة بتقييم النطاق أثناء كتابة التعليمات البرمجية.

```
2 references
public class ScopeTest
{
        2 references
        public string publicTest = "I am a public parameter";
        0 references
        private string privateTest = "I am a private parameter";
        1 reference
        protected string protectedTest = "I am a protected parameter";
        1 reference
        public void RunProgram(){}
        0 references
        private void PrivateProgram(){}
        0 references
        protected void Protected Program(){}
}
```

الشكل 27.31 النطاقات العامة والخاصة والمحمية.

لنقم بإنشاء فئة عامة تسمى **ScopeTest**. بداخلها ثلاث معلمات عامة وخاصة ومحمية، وثلاث طرق عامة وخاصة ومحمية (الشكل 27.30).

إذا قمنا بإنشاء فئة أخرى "توسع" **scopeTest**، تسمى **scopetSextended** (الشكل 27.32)، حينئذٍ يمكننا اختبار نطاق هذه المعلمات والطرق:

```
private class ScopeTestExtended : ScopeTest
{
      0 references
      private ScopeTestExtended(){
          var publicTestExtended = publicTest; //public parameter visible
          var protectedTextExtended = protectedTest; //parent class visible
          //IDE will not allow the use of privateTest string from above class here as not visible
          RunProgram();
          ProtectedProgram();
          //PrivateProgram(); //is not visible
      }
}
```

الشكل 27.32 فئة التمديد الخاصة بـ *scopeTest*.

يتم تعريف هذه الفئة الموسعة على أنها خاصة، ويتم تعريف الطريقة الموجودة بداخلها أيضًا على أنها خاصة. ضمن هذه الطريقة، تكون المعلمات والطرق الوحيدة المرئية عامة ومحمية.

وإذا أنشأنا فئة ثالثة لا تتعلق بأي من الفئتين الأوليين (الشكل 27.33). سيتعين علينا إنشاء مثيل لفئتنا العامة، ولن تكون الفئة الخاصة قابلة للاستنساخ، ولن يتاح الوصول إلا إلى المعلمات والطرق العامة من البداية.

```
private class AnotherClass
{
    0 references
    private AnotherClass()
    {
        var scopeTestObject = new ScopeTest();
        var publicAnotherTest = scopeTestObject.publicTest;
        scopeTestObject.RunProgram();
        //cannot see private or protected parameters or methods here

        var scopeTestExt = new Sco
                                ScopeTest
    }
```

الشكل 27.33 فئة معزولة تسمى *AnotherClass*.

ضمن فئتنا المعزولة، يمكننا "رؤية" فئة **ScopeTest** وإنشاء مثيل لها. يمكننا رؤية المعلمة **publicTest** و**RunProgram** العام. لا يمكننا "رؤية" أي معلمات أو طرق خاصة أو محمية. كما لا يمكننا أيضًا "رؤية" فئة **ScopetSextended** **لإنشاء مثيل لها.**

النطاق هو إجراء أمان رئيسي داخل البرامج لإبقاء المعلمات والطرق تحت السيطرة.

التمرير بالقيمة وبالمرجع

يمكن استخدام المعلمات والمتغيرات لتخزين المعلومات التي لا تتغير أبدًا، والمعروفة باسم الثوابت، أو يتم تمريرها إلى طرق ودوال للاستخدام والمعالجة. وفي معظم الحالات، يتم تمرير المعلمات إلى طرق ودوال "بالقيمة". هذا يعني "خذ قيمة هذه المعلمة فقط، استخدمها ولكن لا يمكن تغييرها ما لم يتم تغيير القيمة خارج الطريقة أو الدالة بعد اكتمال الطريقة أو الدالة".

خذ هذا السيناريو على سبيل المثال، لقد قمت بكتابة تقرير على جهاز الكمبيوتر الخاص بك. وتريد من زميلك مراجعة التقرير وتقديم ملاحظات عليه. يمكنك حفظ نسخة من التقرير وإرسالها بالبريد الإلكتروني إلى زميلك، ليقوموا بالتعليق عليها وإرسالها مرة أخرى لك. أصبح لديك نسختين من التقرير، نسختك الأصلية والنسخة التي تتضمن تعليقات توضيحية. في المثال، قمت بتمرير تقريرك "بالقيمة" إلى زميلك.

وفي التعليمات البرمجية، سيبدو هذا مثل الشكل 27.34.

```
1   var myReport = new Report();
2
3   var annotatedReport = EmailToColleagueAndAnnotateReport(myReport);
4
5   Report EmailToColleagueAndAnnotateReport(Report report)
6   {
7       //actions here to annotate and review report
8       return report;
9   }
```

الشكل 27.34 تمرير *myReport* (تقريري) *بالقيمة*.

هذا هو المثال الأول الذي يوضح بنية الدالة. سنتعرف على هذا لاحقًا في الدليل، لذا في الوقت الحالي تقبل هذه الدالة ببساطة نسخة من التقرير وتعيدها إلى البرنامج الرئيسي.

يقوم السطر 1 بإنشاء *مثيل* لكائن تقرير. هذا هو تقريرنا الأصلي.

يوفر السطر 2 مساحة وبنية لمثال التعليمات البرمجية - إنه ليس ضروريًا ويسمح لك بجعل التعليمات البرمجية قابلة للقراءة عند عرضها.

يقوم السطر 3 باستدعاء **EmailToColleagueAndAnnotateReport** ويمرر تقريرنا بالقيمة.

يمثل السطر 7 تعليقًا يصف بعض الإجراءات المحتملة التي يمكن اتخاذها.

يقوم السطر 8 بإرجاع التقرير الأصلي مرة أخرى إلى البرنامج.

عند الرجوع إلى السطر 3، يتم تخزين نسخة من تقريرنا في **annotatedReport** (التقرير الذي يتضمن تعليقات توضيحية) ولم تتغير البيانات الأصلية داخل **myReport**. ربما تم تغيير البيانات **annotatedReport**.

مراجعة ما تعلمته ✓

ما اصطلاح التسمية المستخدم في القصاصة البرمجية أعلاه؟

يمكن أيضًا تمرير المعلمات والمتغيرات بالمرجع. يعني التمرير بالمرجع تمرير المعلمة الفعلية إلى طريقة أو دالة - أي تغييرات يتم إجراؤها على هذه المعلمة ويتم الاحتفاظ بها خارج الطريقة أو الدالة. دعونا نعيد النظر في السيناريو أعلاه وفي الوقت نفسه نمرر مستندنا بالمرجع. يمكنك تمرير المستند الأصلي إلى زميلك للمراجعة والتعليق. ويقومون بالتعليق على المستند وإعادته إليك. هناك نسخة واحدة فقط من المستند.

في التعليمات البرمجية، سيبدو هذا مثل الشكل 27.35.

```
14    //by reference
15    var myReport = new Report();
16    EmailToColleagueAndAnnotateReport(out myReport);
17
18
19    void EmailToColleagueAndAnnotateReport(out Report report)
20    {
21        report.Annotate();
22    }
```

الشكل 27.35 تمرير myReport (تقريري) بالمرجع

السطر 15 هو مستند التقرير.

يستدعي السطر 16 تعليماتنا لتنفيذها على تقريرنا. لاحظ استخدام الكلمة الرئيسية "out" للإشارة إلى التمرير بالمرجع.

السطر 19 عبارة عن طريقة تطلب من الزميل إضافة تعليق توضيحي للتقرير. الجزء المهم هو الفرق بين الأقواس بعد اسم الطريقة. يتم استخدام الكلمة الأساسية "out" لوضع علامة على كائن التقرير على اعتبار أنه تمرير بالمرجع.

يوضح السطر 18 التقرير الذي تم تمريره.

ستؤدي مراجعة قيمة **myReport** قبل وبعد السطر 16 إلى إظهار التغييرات في القيمة داخله.

التسلسل

لقد قمنا بتغطية التسلسل خلال موضوع النماذج. التسلسلات عبارة عن مجموعات من التعليمات لتنفيذ مهمة. تشكل هذه التسلسلات بنية برنامجك وتعطيه "الشكل". إذا قمت بإنشاء برنامج يتبع التسلسل A -> B -> C -> D، فسوف يتبع هذا التسلسل في كل مرة تقوم فيها بتشغيل هذا البرنامج.

لنكتب برنامجًا بسيطًا ينفذ مجموعة من التسلسلات ثم ينتهي.

```
5        int sum = 0;
6        sum = 1 + 2 + 3 + 4;
7        sum = sum * 10;
8        sum = sum - 1;
```

الشكل 27.36 تسلسل بسيط.

الشكل 27.36 هو برنامج حساب بسيط.

يقوم السطر 5 بإنشاء معلمة تسمى sum (المجموع)، وهي عبارة عن عدد صحيح وتحتوي على القيمة 0.

يحدد السطر 6 قيمة المجموع للإجابة على 4 + 3 + 2 + 1. ما قيمة المجموع؟

يحدد السطر 7 قيمة المجموع على قيمة المجموع مضروبًا في 10. ما قيمة المجموع؟

يحدد السطر 8 قيمة المجموع على قيمة المجموع ناقص 1. ما قيمة المجموع؟

حينئذٍ ينتهي البرنامج. نفذ هذا البرنامج مجموعة من التسلسلات وأنتج مخرجات. ومع ذلك، نحتاج أحيانًا إلى تطبيق المنطق على البرنامج ونطلب منه اتخاذ القرارات. نحن نستخدم الاختيار لاتخاذ تلك القرارات نيابة عنا.

التحديد

التحديد هو "طرح الأسئلة" أثناء البرنامج. تتخذ هذه التقنيات قرارات أثناء تشغيل البرنامج. إنها تسمح للبرنامج بتنفيذ إجراء محدد بناءً على المدخلات التي تلقاها. ثمة ثلاثة أنواع من عبارات التحديد التي سنتناولها ونستخدمها في هذه الوحدة: **IF و...IF ELSE** و**Switch...Case**. تتطلع جميع القرارات المتخذة في عبارات التحديد إلى حل الحساب أو القرار إلى "صواب أو خطأ" أو "نعم" أو "لا" أو "1" أو "0". يعطي هذا القرار قيمة منطقية تقود عملية صنع القرار.

عبارات IF و IF...ELSE

تعمل عبارات IF و IF... ELSE على مبدأ مقارنة القيمة في عبارتها بالصواب أو الخطأ، على سبيل المثال "...If True" القيمة تساوي "صواب"- افعل هذا، وإلا افعل شيئًا آخر. يمكننا تمثيل هذا الاختيار في مثال واقعي:

إذا شعرت بالعطش، فسأشرب بعض الماء، وإلا فلن أشرب بعض الماء.

يظهر نفس المثال في التعليمات البرمجية في الشكل 27.37.

```
internal class IFStatement
{
    0 references
    private static void Main(string[] args)
    {
        string thirsty = args[0];
        if (thirsty == "thirsty")
        {
            DrinkWater();
        }
    }
}
```

الشكل 27.37 عبارة If.

الخطوط من 1 إلى 4 هي بداية برنامجنا. هذا يطلب من جهاز الكمبيوتر إنشاء كائن يسمى **IFStatemation**. أول شيء يقوم به هذا البرنامج هو تشغيل الإجراء الرئيسي مع الخيارات المضافة قبل تشغيله (واحدة أو أكثر من الوسيطات **args**). ولتشغيل هذا البرنامج، سنضيف قيمة 'thirsty' "العطش" إلى معلمة **args**.

العبارة "0 references" مراجع" ليست سطرًا من التعليمات البرمجية، إنها أحد مؤشرات visual studio (المزيد حول هذا لاحقًا في فصل بيئة التطوير المتكاملة "IDE") لعدد الكائنات الأخرى التي تشير إلى هذه الطريقة.

السطر 5 - يأخذ هذا الوسيط الأول (نحسب من 0 في البرمجة) من قائمة الوسيطات الخاصة بقيمة العطش ويضيفها إلى معلمة تسمى 'thirsty' "العطش". وهذا ما يجعل التعليمات البرمجية الخاصة بنا قابلة للقراءة ونقطة مرجعية سهلة لهذه القيمة طوال البرنامج. من الأسهل بكثير كتابة التعليمات البرمجية باستخدام المعلمات المسماة بدلاً من **args[0]** أو **args[10]** أو **args[200]**. سيحتوي هذا الوسيط إما على كلمة "عطشان" أو لاشيء في المعلمة التي تسمى 'thirsty' "عطشان".

السطر 6 - هذا هو عبارة التحديد الخاصة بنا. يستخدم عبارة **IF** للسؤال عما **إذا كانت** قيمة "العطش" تساوي "العطش" مع إعطاء قيمة صواب أو خطأ (تُعرف باسم القيمة المنطقية).

السطر 7 - **{or}** / 'curly bracket' ("قوس معقوف" / "أو") لمحتويات عبارة **IF**. كل شيء داخل الأقواس المعقوفة ينتمي إلى عبارة IF، المعروفة باسم الكتلة. لا يتأثر أي شيء بالخارج بعبارة IF. هذا هو القوس المعقوف الافتتاحي الخاص بنا، وأي سطور حتى القوس المعقوف الختامي هو جزء من عبارة **IF** الخاصة بنا.

السطر 8 - هو الإجراء الذي نريد القيام به لشرب الماء. لا توجد مسافات في الطريقة، ومن ثم تمت كتابتها في صورة **DrinkWater**. تسمح الأقواس الموجودة بعد الاسم لـ**DrinkWater** بأخذ معلمات إضافية للتحكم في طريقتها. وبالمثل، بالنسبة للموضوع أعلاه حول معلمات الكائن، يمكننا تمرير المعلمات إلى الطرق لدفع القرارات داخلها.

السطر 9 - هو القوس المعقوف الختامي، '}'، الذي يخبر البرنامج بأن عبارة **IF** قد انتهت.

يغلق السطران 10 و11 برنامجنا الافتتاحي والأقواس الرئيسية. بعد هذه السطور، لا يوجد شيء آخر للتنفيذ، لذلك ينتهي البرنامج.

وبعد السطر 9 يمكن إضافة عبارات **IF** إضافية. إذا كنت تتخذ قرارات مختلفة بإجراءات مختلفة متنافية، حينئذٍ يمكن إضافة **عبارات ELSE IF** إضافية.

لنفترض على سبيل المثال، أننا إما في درجة حرارة مناسبة، أو شديدة البرودة، أو دافئة جدًا. لا يمكننا أن نكون كلاهما في نفس الوقت، لذا دعونا نكتب ذلك في التعليمات البرمجية باستخدام عبارات "**IF... ELSE**".

المصطلحات الرئيسية

الكتلة: أسطر التعليمات البرمجية التي تم تجميعها معًا.

```
1   internal class IFELSEStatement
2   {
        0 references
3       private static void Main(string[] args)
4       {
5           string temperature = args[0];
6           if (temperature == "too cold")
7           {
8               GetWarm();
9           }
10          else if (temperature == "too warm")
11          {
12              GetCool();
13          }
14          else
15          {
16
17          }
18      }
19  }
```

الشكل 27.38 عبارة If...Else.

تتشابه بداية البرنامج إلى حد كبير جدًا مع البرنامج السابق. لدينا برنامج يسمى **IFELSEStatement**، مع طريقة رئيسية تأخذ الوسيطات (الشكل 27.38). هذه المرة سنضيف درجة حرارتنا إلى الوسيط الأول.

السطر 5 - هذا يحصل على القيمة المعتدلة في معلمة محلية تسمى 'temperature' "درجة الحرارة".

السطر 6 - هذه هي عبارة **IF** الأولى لدينا. على غرار المثال السابق، يطلب البرنامج أن تكون 'temperature' "درجة الحرارة" مساوية لـ'too cold' "شديدة البرودة". إذا كانت درجة الحرارة تساوي "شديدة البرودة"، فانتقل إلى الأسطر 7 و8 و9. إذا كانت درجة الحرارة **لا** *تساوي* "شديدة البرودة"، فانتقل مباشرة إلى السطر 10.

السطر 7 - هذا هو القوس الافتتاحي لعبارة **IF** الأولى.

السطر 8 - هذه تعليمات لتنفيذ الإجراء **GetWarm.**

السطر 9 - القوس الختامي لعبارة IF الأولى.

السطر 10 - هذا هو اتخاذ القرار التالي ضد درجة الحرارة. سيسأل البرنامج عما إذا كانت "درجة الحرارة" تساوي "دافئة جدًا". فإذا كانت درجة الحرارة تساوي "دافئة جدًا"، فانتقل إلى الأسطر 11 و12 و13. وإذا كانت درجة الحرارة **لا** تساوي "دافئة جدًا"، فانتقل مباشرة إلى السطر 14.

السطر 11 - هذا هو القوس الافتتاحي لعبارة **IF** الثانية

السطر 12 - هذه تعليمات لتنفيذ الإجراء **GetCool**.

السطر 13 - هذا هو القوس الختامي لعبارة **IF** الثانية.

السطر 14 - هذا هو تحديدنا النهائي - 'else' "وإلا". إذا كانت جميع عبارات **IF** السابقة "خطأ"، فقم بتنفيذ التعليمات المضمنة.

السطر 15 - القوس الافتتاحي لعبارة "else".

السطر 16 - لا شيء هنا، لا يحدث شيء إذا كانت جميع العبارات الأخرى خطأ.

السطر 17 - القوس الختامي لعبارة "else".

السطران 18 و19 - أغلق البرنامج. حينئذٍ ينتهي البرنامج.

تعتبر عبارات **If...Else** رائعة لعدد صغير من التحديدات المنطقية البسيطة. إلا أنه مع زيادة عدد الخيارات، تصبح كتلة التعليمات البرمجية أقل كفاءة ويتم اكتمالها ببطء. ثمة طريقة أكثر فاعلية لإجراء التحديدات في البرنامج ألا وهي استخدام **Switch... Case**.

عبارة Switch...Case

تتكون عبارة switch...case من عبارة تشغيل البرنامج، على سبيل المثال:

استخدم هذه القيمة "abc" وابحث عن تطابق، ونفذ هذه التعليمات ثم تابع. يبحث البرنامج عن إجابة صواب أو خطأ لـ"هل قيمة التبديل تساوي قيمة الحالة".

لوضع علامة "matches" لقيمة التبديل، نستخدم كلمة "case"، على سبيل المثال:

القيمة هي "abc". في حال كانت القيمة "a"، قم بذلك وقم بإنهاء التبديل. في حال كانت القيمة "b"، قم بذلك وقم بإنهاء التبديل. في حال كانت القيمة "abc"، قم بذلك وقم بإنهاء التبديل. يجب أن تتطابق القيمة الموجودة في بداية عبارة التبديل ('abc') تمامًا مع القيمة المرتبطة بالحالة لتنفيذ التعليمات الصحيحة. ما هي عبارة الحالة التي تمت مطابقتها: الأولى أم الثانية أم الثالثة؟

دعونا نكتب المثال أعلاه في التعليمات البرمجية (الشكل 27.39).

```
1     var switchValue = "ABC";
2     switch (switchValue)
3     {
4         case "A":
5             PrintOutA();
6             break;
7         case "B":
8             PrintOutB();
9             break;
10        case "ABC":
11            PrintOutABC();
12            break;
13    }
```

الشكل 27.39 عبارة Switch-Case (التبديل-حالة).

يقوم السطر 1 بتعريف المعلمة **SwitchValue** ويقوم بتعيينها على "ABC".

يبدأ السطر 2 عبارة التبديل. هذا يخبر جهاز الكمبيوتر بالتحرك على طول الحالات الموجودة أسفله ومقارنة قيمته بالقيمة المرتبطة بقيمة الحالة.

السطر 3 عبارة عن قوس معقوف يفتح كتلة التعليمات البرمجية لعبارة التبديل.

السطر 4 هو عبارة الحالة الأول. يخبر الكمبيوتر أن هذه الكتلة من التعليمات البرمجية هنا ستعمل إذا كانت قيمة التبديل تساوي "A". تم تعيين قيمة التبديل على "ABC" حتى لا يتم تشغيله والانتقال إلى العبارة التالية. "ABC لا تساوي A" وبالتالي فإن الإجابة خطأ.

السطر 5 هو الإجراء الخاص إذا تمت مطابقة هذه الحالة وتشغيلها.

يخبر السطر 6 جهاز الكمبيوتر بالتوقف عن البحث عن القيم التي يجب مطابقتها في هذه الكتلة. تعني "break" التوقف عما تفعله في كتلة التعليمات البرمجية هذه. تعمل فواصل الأوامر لمجموعة متنوعة من تقنيات البرمجة المختلفة وستظهر في مواضيع أخرى في هذا الدليل. إنها "وقفة صارمة" لكتلة من التعليمات البرمجية، مثل إخبار البرنامج بـ"أوقف ما تفعله وامض قدمًا، من فضلك".

السطر 7 هو عبارة الحالة التالية. كما هو الحال مع الأسطر 4 و5 و6، لن يتم تشغيل هذه الحالة. لا تتطابق قيمة التبديل مع عبارة الحالة - "ABC لا تساوي B" وبالتالي فإن الإجابة خطأ.

السطران 8 و9 يشبهان السطران 5 و6.

السطر 10 هو عبارة الحالة النهائية لدينا. قيمة الحالة هي "ABC"، وقيمة التبديل لدينا هي "ABC". لدينا تطابق! "ABC" تساوي "ABC" مما يؤدي إلى نتيجة *صواب*.

السطر 11 هو الإجراء الذي سيتخذه البرنامج. سيتم طباعة "ABC" على وحدة التحكم أو الشاشة.

يخبر السطر 12 البرنامج بإيقاف ما يفعله في هذه الكتلة والمضي قدمًا. بالنسبة لهذا البرنامج، إنها نهاية البرنامج. وبالنسبة للبرامج الأخرى، قد يتم اتخاذ قرارات أو إجراءات أخرى.

كانت أمثلة البرامج وكتل التعليمات البرمجية حتى الآن مفيدة لمعالجة القرار الفردي، ولكن ماذا لو كنت تريد تنفيذ إجراء مرارًا وتكرارًا في تطبيق واحد دون عمليات بدء وإيقاف متعددة للبرنامج؟ يمكننا استخدام كتل التعليمات البرمجية المتكررة التي تسمى التكرارات لتنفيذ هذا النوع من الإجراءات.

التكرار

يسمح التكرار بتكرار عدة خطوات لتبسيط الإجراء العام. يشار إليه باسم "حلقة تكرارية" في البرمجة. فيما يلي أنواع التكرار التي سنتناولها في هذه الوحدة:

- **الحلقات التكرارية While وDo-While**
- **الحلقات التكرارية For وFor Each**

الحلقات التكرارية While وDo-While

على الرغم من أنها تبدو متشابهة في الاسم، إلا أنها تنطوي على اختلاف أساسي. دعونا نلقي نظرة على الحلقة التكرارية **While** أولاً. كما هو الحال مع التحديد، يستخدم كلاهما شرطًا منطقيًا للتحكم في تنفيذه. من الأمثلة الواقعية على حلقة While التكرارية عندما يكون الجو باردًا، ارتدِ معطفًا. ستنتهي هذه الحلقة التكرارية عندما لا يكون الجو باردًا. يفرض المنطق البولياني الخاص بنا أن **"بارد = صواب" و"ليس باردًا = خطأ".**

بدون **حلقة While** التكرارية، يجب أن تبدو التعليمات البرمجية الخاصة بنا مثلما هو موضح في الشكل 27.40.

```
int temperature = 0;
int isNotColdTemperature = 5;
bool isCold = temperature < isNotCold Temperature;
if (isCold){
    WearCoat();
}
temperature = temperature + 1;
isCold = temperature < isNotColdTemperature;
if (isCold){
    WearCoat();
}
temperature = temperature + 1;
isCold = temperature < isNotColdTemperature;
if (isCold){
    WearCoat();
}
temperature = temperature + 1;
isCold = temperature < isNotColdTemperature;
if (isCold){
    WearCoat();
}
temperature = temperature + 1;
isCold = temperature < isNotColdTemperature;
if (isCold){
    WearCoat();
}
temperature = temperature + 1;
isCold = temperature < isNotColdTemperature;
if (isCold){
    WearCoat();
}
else {
    RemoveCoat();
}
```

الشكل 27.40 التعليمات البرمجية بدون الحلقة التكرارية whileloop.

هذه ليست تعليمات برمجية فعالة وقابلة للتطوير. وفيما يتعلق بإجراء فحص بسيط يبلغ 5 درجات، يلزم وجود 34 سطرًا من التعليمات البرمجية. لا يمكنك تغيير الهدف ودرجات حرارة الأولية دون إضافة المزيد من كتل التعليمات البرمجية لكل زيادة. تتمثل أفضل طريقة في استخدام التكرار لتكرار التعليمات البرمجية المتكررة.

الحلقة التكرارية While

من أجل استخدام الحلقة التكرارية **While**، يجب أن نعطي الحلقة الخاصة بنا "شرط الخروج". فإذا لم يتم الخروج من الحلقة التكرارية مطلقًا، فسنكون عالقين في حلقة لا نهائية.

المصطلحات الرئيسية

الحلقة اللانهائية: جزء من التعليمات البرمجية لا يحتوي على شرط خروج، وبالتالي يتكرر إلى الأبد.

```
1    int temperature = -10;
2    int isNotColdTemperature = 20;
3    bool isCold = temperature < isNotColdTemperature;
4    while (isCold)
5    {
6        WearCoat();
7        temperature = temperature + 1;
8        isCold = temperature < isNotColdTemperature;
9    }
10   RemoveCoat();
```

الشكل 27.41 الحلقة التكرارية "While".

دعونا نقيم هذه التعليمات البرمجية سطرًا بسطر. لقد بدأنا في بناء كتل أكثر تعقيدًا من التعليمات البرمجية.

السطر 1 - يوضح درجة الحرارة الحالية. إنها 10-.

السطر 2 - يوضح درجة الحرارة التي يجب أن تكون عليها قبل ألا يصبح الجو باردًا بعد الآن. هذه القيمة هي 20.

السطر 3 - يقرر البرنامج ما إذا كانت درجة الحرارة الحالية باردة أم لا. هل درجة الحرارة الحالية أدنى أم أقل من درجة الحرارة المطلوبة لخلع المعطف؟ في بداية البرنامج، نعم هو كذلك. **IsCold** يساوي *صواب*.

السطر 4 - هذه هي بداية حلقتنا التكرارية while. ينص البرنامج على ما يلي: بينما يكون الجو باردًا، قم بتنفيذ الإجراءات الموجودة بين قوسين.

السطر 5 - هو افتتاحية الحلقة التكرارية while.

السطر 6 - يتم تنفيذ إجراء **WearCoat**.

السطر 7 - تزداد درجة الحرارة بمقدار 1.

السطر 8 - يتم إعادة تقييم المعلمة **IsCold**. في التمريرة الأولى، درجة الحرارة الحالية ليست 9-، ودرجة الحرارة المستهدفة لا تزال 20. لا تزال المعلمة **IsCold** تعادل "صواب".

السطر 9 - هو القوس النهائي للحلقة التكرارية. بدلاً من التمرير إلى السطر 10، يظل شرط حلقة while التكرارية صواب. يعود البرنامج إلى السطر 3، ويقيّم الشرط بين قوسين حول **(while (isCold**، وبحسب "true" (صواب) ويقوم بتشغيل الكتلة مرة أخرى.

في كل مرة تمر عبر الكتلة تزداد درجة الحرارة الحالية بمقدار 1. وفي النهاية، تصبح درجة الحرارة الحالية مساوية لـ20. يقوم البرنامج بتقييم شرط **IsCold** في السطر 3، وأصبحت معلمة **IsCold** الآن تساوي خطأ. تكتمل حلقة while التكرارية ويدخل البرنامج السطر 10.

السطر 10 - يتم تشغيل إجراء **RemoveCoat**.

حينئذٍ ينتهي البرنامج.

في المثال أعلاه، لن يتم تشغيل طريقة **WearCoat** وزيادة درجة الحرارة إلا إذا كان الشرط في بداية حلقة while true (صواب). إذا بدأ البرنامج وكانت المعلمة **isCold** خطأ، فلن يتم تشغيل طريقة **WearCoat** أبدًا، وسيتم تشغيل **RemoveCoat على الفوربدلاً** من ذلك. وهذا ما يتسبب في وضع البرنامج في حالة صعبة. من المفترض في بداية البرنامج عدم ارتداء معطف. يمكن أن تتسبب مطالبة البرنامج بإزالة معطف غير موجود في حدوث مشكلة. يحتاج البرنامج إلى استدعاء **WearCoat** قبل أن يتمكن من **RemoveCoat**، حتى نتمكن من استخدام عبارة do-while لتسهيل هذا الأمر.

الحلقة التكرارية Do-While

ستقوم الحلقة التكرارية **Do-While** بتنفيذ إجراء ما، قبل التحقق مما إذا كان شرط *while* صوابًا. عكس حلقة **While** التي تتحقق مما إذا كان الشرط *صوابًا* قبل تنفيذ الإجراء.

دعونا نكرر المثال أعلاه، ولكن باستخدام Do While. يحتاج البرنامج إلى استدعاء **WearCoat** قبل التحقق من درجة الحرارة وفي النهاية **RemoveCoat** (الشكل 27.42).

```
1   int temperature = -10;
2   int isNotColdTemperature = 20;
3   bool isCold = temperature < isNotColdTemperature;
4   do
5   {
6       WearCoat();
7       temperature = temperature + 1;
8       isCold = temperature < isNotColdTemperature;
9   }
10  while (isCold);
11  RemoveCoat();
```

الشكل 27.42 مثال على التعليمات البرمجية Do-While.

تتكون عبارة **Do-While** من إجراء *Do* (**WearCoat**، زيادة درجة الحرارة، إعادة تقييم **IsCold**) ثم تتحقق من شرط **IsCold** لتحديد ما إذا كنت تريد تكرار تدفق البرنامج أو الاستمرار فيه.

الحلقة التكرارية For وFor Each

تشمل أنواع التكرار الأخرى **For** و**For Each**. يعمل هذان النوعان بطريقة متشابهة جدًا ولكن مع تطبيقات مختلفة جدًا. تُستخدم حلقة For التكرارية للتكرار من خلال قائمة أو مجموعة من الكائنات بينما يكون الشرط المنطقي صواب. عادةً ما بنية الحلقة التكرارية for هي،

For (قم بتعيين فهرس البداية وقيمته، وشرط تكرار الحلقة، وكيفية تغيير فهرس البداية في كل عملية تكرار).

على سبيل المثال For (int a = 0; a < 20; a++)

++ أو -- تزيد أو تنقص المعلمة التي تسبقها.

دعونا نلقي نظرة على حلقة for تكرارية بسيطة لإخراج كلمة إلى Console (وحدة تحكم) بناءً على نتيجة الحساب. سيحسب البرنامج من 0 إلى 20، عندما يصل إلى رقم قابل للقسمة بالتساوي على 3، وستكون المحصلة كلمة "FIZZ". عندما تصل إلى رقم قابل للقسمة بالتساوي على 5، فستظهر كلمة "BUZZ". وعندما تصل إلى رقم قابل للقسمة بالتساوي على 3 و5، فستظهر كلمة "FIZZBUZZ". العامل البرمجي الأكثر ملاءمة لاستخدامه في هذا الحساب هو modulo (باقي القسمة).

المصطلحات الرئيسية

الزيادة: تُعرف زيادة الرقم بمقدار واحد بالزيادة. ويمكن كتابة هذا في صورة a = a + 1 أو a++. يُعرف ++ بعد المتغير باسم الزيادة اللاحقة.

النقصان: يُعرف تقليل الرقم بمقدار واحد بالنقصان. ويمكن كتابة هذا في صورة a = a – 1 أو a--. يُعرف -- بعد المتغير باسم النقصان اللاحق.

دعونا نخطط لهذا الحلقة التكرارية باستخدام الكود الزائف.

بالنسبة لعدد البداية = 1، يكون عدد البداية أقل من 20، أضف 1 إلى عدد البداية، ومن ثم

وإذا كان عدد البداية يساوي 0، فلا تفعل شيئًا - تابع إلى التكرار التالي.

وإذا لم ينتج عدد البداية المقسوم على 3 أي باقي، فقم بإخراج Fizz.

وإذا لم ينتج عدد البداية المقسوم على 5 أي باقي، فقم بإخراج Buzz.

ارجع إلى حلقة for التكرارية للتعرف على التكرار التالي.

لننتج تعليمات برمجية لحل هذه المسألة (الشكل 27.43).

```
1    int maxNumber = 20;
2    for (int i = 1; i < maxNumber; i++)
3    {
4        if (i % 3 == 0)
5        {
6            Console.Out.Write("FIZZ");
7        }
8        if (i % 5 == 0)
9        {
10           Console.Out.Write("BUZZ");
11       }
12       Console.Out.WriteLine();
13   }
```

الشكل 27.43 نموذج لبرنامج Fizz-Buzz.

السطر 1 - يحدد **maxNumber** الذي يجب أن تشغله حلقة for التكرارية.

السطر 2 - عبارة *For*. تقوم الأجزاء الثلاثة من حالة السطر بتعيين رقم البداية "i" على 0، بينما تكون "i" أقل من حلقة العدد الأقصى من خلال هذه التعليمات البرمجية، مما يؤدي إلى زيادة "i" بمقدار 1 في كل مرة.

يقوم السطر 4 بتنفيذ عبارة **IF** الأولى. *إذا كانت* قيمة باقي قسمة '**i**' *على 3* تساوي 0 (لا يوجد باقي)، فانتقل إلى السطر 6.

السطر 6 - إخراج "FIZZ" إلى console (وحدة التحكم).

يقوم السطر 8 بتنفيذ عبارة **IF** الثانية. *إذا كانت* قيمة باقي قسمة '**i**' *على 5* تساوي 0، فانتقل إلى السطر 10.

السطر 10 - إخراج "BUZZ" إلى console (وحدة التحكم).

السطر 12 - ابدأ سطرًا جديدًا في console (وحدة التحكم).

يعمل هذا جيدًا للأرقام المتسلسلة من 0 إلى 20. ماذا لو كانت أرقامنا في مصفوفة أو قائمة بترتيب عشوائي؟ يمكننا استخدام حلقة For Each لإنتاج نفس النتيجة.

تتكرر حلقة For *each* من خلال *كل* عنصر في قائمة أو مصفوفة. في كل مرة تتكرر فيها خلال هذه المجموعة، فإنها تستخدم العنصر التالي.

دعونا نكرر برنامج FIZZ-BUZZ الخاص بنا ولكن مع قائمة عشوائية من الأعداد الصحيحة (الشكل 27.44).

```
int[] numbers = new int{3,7,2,4,8,4,55,5,4,3,21,8,3,15,344,223,77,4,3,386,4};
foreach(int number in numbers)
{
    if (number % 3 == 0)
    {
        Console.Out.Write("FIZZ");
    }
    if (number % 5 == 0)
    {
        Console.Out.Write("BUZZ");
    }
    Console.Out.WriteLine();
}
```

الشكل 27.44 Fizz Buzz مع إضافة أرقام عشوائية.

السطر 1 هو مصفوفة الأعداد الصحيحة "العشوائية".

السطر 2 هو عبارة **For Each**، وهي تنص على كل عدد صحيح، مع كل مثيل يسمى "الرقم" في مصفوفة الأرقام يقوم بتنفيذ الإجراءات داخل الأقواس المعقوفة.

السطر 4 إلى 12 ينفذ نفس حساب *FizzBuzz* كما في مثال *For* سابقًا.

مراجعة ما تعلمته

قم بإنشاء نسختك الخاصة من هذا البرنامج. وبالنسبة للقيم التي تقسم بالتساوي على 2، قم بإخراج كلمة من اختيارك وبالنسبة للقيم التي تقسم بالتساوي على 7، قم بإخراج كلمة مختلفة من اختيارك.

النشاط

لتوضيح حلقة for وforeach التي تعمل معًا، يعرض المثال أدناه برنامجًا متعدد النماذج. يستخدم هذا المثال عناصر إجرائية وموجهة نحو الكائنات وقائمة على الأحداث.

```
internal class CupOfTea{
    1 reference
    public string Flavour;
    5 references
    public int AmountOfTea;
    1 reference
    public void Drink(){
        //drink tea
        //let program know has been drunk
        //loop through each amount of tea (FOR LOOP)
        for(var i = AmountOfTea; i == 0; i--)
        {
            DrinkSipOfTea();
        }
        AmountOfTea = : 0;
        TeaDrunk(Flavour);
    }
    4 references
    public event EventHandler TeaDrunk;
}
```

النشاط *تابع*

```
internal class DrinkLoadsOfTea
{
        4 references
        private List<Cup0fTea> CupsOfTea = new List<CupOfTea>();
        0 references
        public void Main(string[] args)
        {
                //build our tea list
                var cupOfTea1 = new CupOfTea();
                cupOfTea1.AmountOfTea = 80;
                cupOfTea1.TeaDrunk += TeaDrunk;
                CupsOfTea.Add(cupofTea1);

                var cupOfTea1 = new CupOfTea();
                cupOfTea1.AmountOfTea = 100;
                cupOfTea2.TeaDrunk += TeaDrunk;
                CupsOfTea.Add(cupOfTea2);

                var cupOfTea3 = new Cup0fTea();
                cupOfTea3.AmountOfTea = 50;
                cupOfTea3.TeaDrunk += TeaDrunk;
                CupsOfTea.Add(cupOfTea3);

                //loop through each (FOREACH) cup of tea
                foreach(CupOfTea cupOftea in CupsOfTea)
                {
                        cupoftea.Drink();
                }

        }

        3 references
        private EventHandler TeaDrunk(string Flavour)
        {
                Console.Out.WriteLine(Flavour + " tea has been drunk.");
        }
}
```

الشكل 27.45 برنامج متعدد النماذج.

انسخ هذه التعليمات البرمجية وقم بالتعليق عليها لتسليط الضوء على موضوعات الوحدة التالية:

- تسليط الضوء على كتلة من التعليمات البرمجية الإجرائية
- تسليط الضوء على كائن
- تسليط الضوء على فئة
- تسليط الضوء على حدث مخصص
- قم بالتعليق على موقع حلقة For وموقع حلقة For **Each**

أعد كتابة حلقة For Each لاستخدام حلقة **For** بدلاً من ذلك.

(أ3) أدوات التطوير

المجمعات والمحولات والمفسرات البرمجية

يمكن كتابة برامج الكمبيوتر بلغات مختلفة إما لغات HIGH LEVEL (عالية المستوى) أو LOW LEVEL (منخفضة المستوى). تركز هذه الوحدة على الكتابة بلغة برمجة عالية المستوى. يحتاج كلا النوعين إلى التحويل إلى لغة الآلة حتى يتمكن معالج الكمبيوتر من استخدام التعليمات الموجودة في البرنامج. يتم ذلك باستخدام *المجمعات والمحولات والمفسرات* البرمجية.

المصطلحات الرئيسية

البت: رقم ثنائي (يمثله 0 أو 1) وأصغر وحدة بيانات يمكن للكمبيوتر معالجتها وتخزينها.

يتكون البايت من 8 بت.

يأخذ المجمّع البرمجي التعليمات الأساسية ويحولها إلى أنماط من وحدات **البت** التي يمكن لمعالج الكمبيوتر استخدامها تُعرف أنماط البتات هذه أحيانًا باسم لغة التجميع أو المجمّع البرمجي. ويُعرف باسم "المستوى المنخفض" وهو مشابه جدًا للغة الأصلية للمعالج.

المحول البرمجي هو "مترجم" للغات البرمجة عالية المستوى مثل Java أو C++®. تعمل هذه المحولات البرمجية على تحويل التعليمات إلى لغة الآلة في شكل "برنامج قابل للتنفيذ" دفعة واحدة، بشرط أن تكون التعليمات البرمجية خالية من الأخطاء. ستسلط معظم المحولات البرمجية الضوء على الأخطاء في التعليمات البرمجية أثناء عملية التحويل البرمجية والتي يجب حلها قبل أن يتم "ترجمة" التعليمات البرمجية للبرنامج.

يقوم المفسر البرمجي بتحويل تعليمات البرمجة عالية المستوى إلى تعليمات برمجية للآلة سطرًا بسطر. عند العثور على خطأ، يتوقف البرنامج ويخرج على السطر الخاطئ. فإذا لم يتم العثور على أخطاء، يستمر المفسر البرمجي حتى الوصول إلى نهاية التعليمات البرمجية.

بيئات التطوير المتكاملة

تحتوي العديد من بيئات التطوير المتكاملة (IDE) على المحولات والمجمعات والمفسرات البرمجية. بيئة التطوير المتكاملة (IDE) هي "المجمع الموحد" لتطوير برنامج باللغة التي اخترتها. ستركز هذه الوحدة على Visual Studio، وهو بيئة التطوير المتكاملة لإطار لغات .NET. يتضمن هذا البرنامج محولًا برمجيًا ومحررًا للتعليمات البرمجية وأدوات بناء ومجموعات تطوير البرامج (SDKs) والمكتبات ومصححات الأخطاء والتدرج خلال الشيفرة البرمجية. هذه هي جميع الأدوات اللازمة لبناء برامج ناجحة.

محررات التعليمات البرمجية

محررات التعليمات البرمجية عبارة عن برامج تسهل كتابة وتحرير تعليمات البرمجة. وفي الغالب كان المبرمجون قبل محررات التعليمات البرمجية يكتبون التعليمات البرمجية في برامج تحرير النصوص مثل notepad أو vim. تتمحور معظم بيئات التطوير المتكاملة حول محرر التعليمات البرمجية مع ميزات مضمنة مثل تمييز الأخطاء والنصائح لكتابة تعليمات برمجية فعالة وخالية من الأخطاء. يتم تمييز الأخطاء أثناء كتابة التعليمات البرمجية لمساعدة المطور.

المحولات البرمجية وأدوات البناء.

كما ذكرنا سابقًا، يقوم المحول البرمجي بتحويل تعليمات البرمجة إلى أنماط من التعليمات التي يمكن لمعالج الكمبيوتر فهمها. تحتوي بيئات التطوير المتكاملة على محولات برمجية بداخلها، مما يبسط عملية التحويل البرمجي إلى ما يعرف باسم "Build" و"Run". يؤدي الضغط على زر "Build" داخل بيئة التطوير المتكاملة إلى تحويل التعليمات البرمجية إلى برنامج قابل للتنفيذ، في مجلد ذي صلة بالتعليمات البرمجية المصدر. يقوم زر "Run" بنفس عملية "Build" فضلًا عن أنه يقوم أيضًا بتنفيذ البرنامج الناتج داخل بيئة التطوير المتكاملة.

حزمة تطوير البرامج

حزمة تطوير البرامج، أو SDKs، هي أدوات لتطوير البرامج. وهي عبارة عن حزم من البرامج أو مصححات الأخطاء أو المحولات البرمجية أو المكتبات التي تساعد المطور على إضافة وظائف حزمة تطوير البرامج إلى برنامجه. قد تتضمن حزمة تطوير البرامج النموذجية ما يلي:

- المحولات البرمجية
- عينات التعليمات البرمجية
- مكتبات التعليمات البرمجية أو إطار العمل
- أدوات الاختبار والتحليل
- الوثائق
- مصححات الأخطاء.

عند بدء كتابة التعليمات البرمجية بلغات عالية المستوى مثل C# أو Java، ستستخدم حزمة تطوير البرمجيات. تمنحك حزمة Java Developer Kit (JDK)® مجموعة من الأدوات الأساسية لكتابة البرامج بلغة Java، بينما يتيح لك إطار العملNET. Framework كتابة برامج بمجموعة أساسية من الأدوات بلغاتNET. مثل C# أو Visual Basic. تعد حزم تطوير البرامج جزءًا أساسيًا من تطوير البرامج. قد تتضمن بعض استخدامات حزم تطوير البرامج ما يلي:

- توفر حزم تطوير البرامج التحليلية بيانات حول إجراءات المستخدم وسلوكياته ومساراته، مثل Google®
- توفر حزم تطوير البرامج الإعلانية إعلانات المستخدم والإعلانات العامة لتطبيقات الأجهزة المحمولة والبرامج الأخرى، مثل Facebook® أو Amazon® أو Microsoft
- توفر حزم تطوير البرامج الخاصة بالدفع خدمات الدفع مثل ApplePay® أو GooglePay® أو AmazonPay®.

لماذا تستخدم حزم تطوير البرامج؟

توفر حزم تطوير البرامج للمطورين الكثير من الوقت والمال. كما أنها تضيف ميزات "مجربة ومختبرة" إلى البرنامج، مع القليل من تعليمات البرمجة الإضافية. فيما يلي المزايا الخاصة بها:

- الوصول إلى الميزات وكتل التعليمات البرمجية لتطوير البرامج. على سبيل المثال، توفر حزمة تطوير البرامج للتجارة الإلكترونية جميع الأشياء التي قد تحتاجها لتطبيق ما مثل المفضلات وعربة التسوق والسداد مع الخروج والمنتجات
- عبارة عن عمليات تكامل فعالة وأسرع، من خلال توفير عمليات مبسطة للحصول على المعلومات المطلوبة
- تقليل أوقات التطوير، وإطلاق المنتجات وإدخالها إلى السوق بشكل أسرع وأكثر كفاءة
- سهولة الدعم والتوثيق، حيث تأتي حزم تطوير البرامج مع أدلة المساعدة والمستندات المرجعية. ستحتوي حزم تطوير البرامج المعروفة والمثبتة على إجابات للأسئلة الشائعة في منتديات الإنترنت المستخدمة بشكل شائع من قبل المطورين، مثل StackOverflow
- عبارة عن حزم تطوير برامج فعالة من حيث التكلفة ومجانية ومدفوعة تسمح للمطورين بتقدير الميزانية والبقاء في حدود الميزانية الخاصة بالمشاريع.

المكتبات

المكتبات عبارة عن مجموعات من التعليمات البرمجية التي توفر وظائف إضافية. وعلى غرار حزم تطوير البرامج، فإنها يمكنها إضافة ميزات ووظائف جديدة يمكن للمطور استخدامها ودمجها في البرامج. ورغم ذلك، فإنها مجرد تعليمات برمجية وقد تأتي أو لا تأتي مع الوثائق.

مصححات الأخطاء

مصححات الأخطاء عبارة عن أدوات برمجية أو برامج تساعد المطور في تحديد الأخطاء ومراقبتها في التعليمات البرمجية الخاصة بالبرنامج. كما أنها تمكّن المطور من تتبع كل سطر من البرنامج واستقصاء كل عنصر من عناصر التعليمات البرمجية. فيما يلي بعض مصححات الأخطاء الشائعة:

- WinDbg®
- Visual Studio Debugger®
- Eclipse Debugger®
- Firefox Javascript Debugger®.

يحتوي كل مصحح أخطاء على برامج ولغات متوافقة خاصة به. في هذه الوحدة، يتم تطوير البرامج في Visual Studio وبالتالي يتم استخدام مصحح الأخطاء Visual Studio Debugger المضمن.

التدرج خلال الشيفرة البرمجية

عند تصحيح أخطاء البرنامج، يمكن للمطور إضافة "نقاط التوقف" إلى التعليمات البرمجية الخاصة به. تخبر هذه النقاط الموجودة في البرنامج مصحح الأخطاء بـ"التوقف هنا". بدون نقاط التوقف، سيتم تشغيل مصحح الأخطاء من خلال البرنامج من البداية إلى النهاية دون أي انقطاع. يضيف المطورون نقاط توقف حول طرق أو معلمات محددة و"يتنقلون" عبر التعليمات البرمجية سطرًا بسطر لتنفيذ البرنامج. يتضمن Visual Studio، وكذلك العديد من مصححات الأخطاء الأخرى، قدرات *Step In* و*Step Out* و*Step Over* و*Run*.

عندما يتم تصحيح أخطاء البرنامج ويتم "إيقافه مؤقتًا" حاليًا عند تنفيذ اسم الطريقة، فإن **STEP IN** ستنتقل إلى التعليمات البرمجية للطريقة.

وعندما يتم إيقاف البرنامج مؤقتًا داخل الطريقة، تنتقل **STEP OUT** خارج الطريقة ويعود إلى التعليمات البرمجية التي تسمى الطريقة.

وعندما يتم إيقاف البرنامج مؤقتًا على سطر برنامج آخر، ستنتقل **STEP OVER** إلى السطر التالي في الكتلة الحالية.

وللانتقال إلى نقطة التوقف التالية أو متابعة تشغيل البرنامج، سيضغط المطور على **RUN**.

الأخطاء ومعالجة الأخطاء

البرامج لا تخلو من الأخطاء أبدًا. فحتى عمالقة تطوير البرمجيات لا يزالون يواجهون أخطاء أو "استثناءات" في برامجهم. هل سبق لك أن تعرضت لـ"تعطل" جزء من برنامج أو لعبة بسبب خطأ؟ عندما يواجه أحد البرامج خطأ، هناك طريقتان للمتابعة:

- طريقة يتم التعامل معها والتحكم فيها
- طريقة غير معالجة.

من الممارسات الجيدة توقع الأماكن التي قد تحدث فيها الاستثناءات في برامجك ومنع ظهور الأخطاء. هذا ليس دائمًا سهلًا كما يبدو، وتحتاج الأخطاء الأكثر شيوعًا إلى "معالجة". تم بناء "معالجة الاستثناءات" في C# على أربع كلمات أساسية: **try** و**catch** و**finally** و**throw**.

أخطاء بناء الجملة ووقت التشغيل.

تتضمن الأنواع الأخرى من الأخطاء أخطاء بناء الجملة ووقت التشغيل. يتم تمييز أخطاء بناء الجملة أثناء ترميز البرنامج. عادةً ما تقوم بيئات التطوير المتكاملة بتمييز واقتراح تصحيحات لهذه الأنواع من الأخطاء. معظم لغات البرمجة حساسة لحالة الأحرف، لذا يمكن أن تتسبب الأحرف الكبيرة والأخطاء الإملائية في حدوث أخطاء في بناء الجملة أو أحرف غير صالحة أو نهايات الأسطر المفقودة (الفاصلة المنقوطة، ';').

معالجة الأخطاء

"للتعامل" مع كتلة من التعليمات البرمجية ومنعها من التأثير على تشغيل البرنامج، يمكنك تغليفها في "try/catch". يفترض الغلاف "try" (تجربة) تشغيل هذه التعليمات البرمجية و"catch" (اصطياد) أي أخطاء قد تحدث. يتم عرض كتلة try/catch العامة أدناه:

```
try
{
        int a = null;
        int b = 2;
        int c = a.Equals(b);
}
catch(Exception ex)
{
        //error caught here
}
```

الشكل 27.46 حاول Try وcatch (التجربة والاصطياد).

في الشكل 27.46، تمت كتابة التعليمات البرمجية عمدًا لإظهار خطأ. سيتم تخزين تفاصيل الخطأ في معلمة الاستثناء المسماة "ex" وستحتوي على "Null Reference Exception". يظهر هذا لأنه في السطر 5 يتم طلب إجراء على معلمة فارغة. "a" فارغة لذا لا يمكن تنفيذ أي إجراءات عليها.

أنواع الاستثناءات الشائعة

هناك العديد من أنواع الاستثناءات المختلفة في برامج C#، وفيما يلي الأنواع الأكثر شيوعًا:

- **IOException**، يظهر عند وجود أخطاء في دوال Input/Output (الإدخال/الإخراج)، على سبيل المثال محاولة فتح ملف لا يمكن العثور عليه
- **IndexOutOfRangeException**، يظهر عند الإشارة إلى فهرس من مصفوفة أكبر من نطاق المصفوفة، على سبيل المثال محاولة الإشارة إلى عنصر في الفهرس 10 في مصفوفة تحتوي على 5 عناصر
- **NullReferenceException**، يظهر عند تنفيذ عملية على عنصر لا ينبغي أن يكون فارغًا ولكنه، على سبيل المثال، يظهر نموذج التعليمات البرمجية الخاص بنا أعلاه استثناءً مرجعيًا فارغًا
- **DividebyZeroException**، يظهر عند قسمة الرقم على صفر، على سبيل المثال حساب متوسط القيمة برمجيًا على مصفوفة فارغة من القيم.
- **InvalidCastException**، يظهر عندما يتم تحويل نوع إلى نوع آخر، على سبيل المثال، تحويل عدد صحيح إلى سلسلة نصية (الشكل 27.47)
- **OutOfMemoryException**، يظهر عند نفاد ذاكرة الكمبيوتر التي يمكن تخصيصها، على سبيل المثال، تحاول إنشاء عدد صحيح بقيمة أكبر مما يمكن تخزينه
- **StackOverflowException**، يظهر عندما يتكرر البرنامج كثيرًا، على سبيل المثال، ستؤدي الحلقة اللانهائية إلى حدوث هذا الاستثناء.

```
int number = 5;
string numberStr = (string) number;
```

الشكل 27.47 استثناء تحويل غير صالح.

النشاط

```
1    int i = 100
2    IF (i > 1)
3    {
4        §dfgdfgsdfg
5    }
```

الشكل 27.48 نشاط خطأ في التعليمات البرمجية الجزء 1.

ما هي المشكلات المتعلقة بنموذج التعليمات البرمجية أعلاه؟

أخطاء وقت التشغيل هي المشكلات التي قد تبدو مناسبة لبيئة التطوير المتكاملة أثناء تطوير التعليمات البرمجية ولكنها تسبب مشكلات عند تشغيل البرنامج.

ما الاستثناء الذي ستطرحه الأسطر التالية من التعليمات البرمجية في Runtime؟ لاحظ أن استثناء وقت التشغيل يمنع البرنامج من متابعة الاستثناء، لذا فإن الاستثناء الأول الذي تم العثور عليه ولم يتم معالجته سيؤدي إلى إنهاء البرنامج. ما الاستثناء المحتمل الذي من المحتمل أن يتم تجاهله؟

```
int a = null;
int b = 100;
var c = a * 100;
var d = c / a;
```

الشكل 27.49 نشاط خطأ في التعليمات البرمجية الجزء 2.

إذا تم تعيين 'a' على 0، فهل سيؤدي ذلك إلى حل استثناءات البرنامج؟

ممارسة التقييم هدف التعلم (أ)

دراسة الأدوات والتقنيات المستخدمة في تطوير برنامج

لقد تم تعيينك كمبرمج متدرب في شركة ميكانيكا جوية في الأردن لإجراء ترقية على أحد أنظمة الكمبيوتر الخاصة بها. وقبل الشروع في المشروع، يرغبون في كتابة سلسلة من البرامج الصغيرة المشروحة والوظيفية لإظهار فهمك ومعرفتك ببنيات البرمجة الأساسية. وبالنسبة لكل برنامج، قم بشرح وتقييم استخدام كل أداة أو تقنية واقتراح سبب اختيار تقنية على أخرى. ستوضح معرفتك في تقرير يحتوي على لقطات شاشة لبرامجك وتعليماتك البرمجية، أو عرض تقديمي يوضح تشغيل برامجك أثناء مناقشة نتائجك مع مديرك.

تابع هدف التعلم (أ)

- اكتب برنامجًا **إجرائيًا** يحسب من 1 إلى 100 ويقوم بإجراء عمليات حسابية مختلفة.
- اكتب برنامجًا **موجهًا نحو الكائنات** يرسم أشكالًا مختلفة على الشاشة.
- اكتب برنامجًا **قائمًا على الأحداث** يطالب المستخدم بإدخال تفاصيل الاتصال الخاصة بالشركة.

بالنسبة لكل برنامج، تأكد من إظهار استخدام جميع الأدوات والتقنيات التالية التي يتناولها هذا الموضوع.

لاختتام تقريرك أو عرضك التقديمي، ناقش كيفية استخدامك لأدوات التطوير. ناقش الغرض وتجربتك الخاصة للأدوات التي تم تناولها في هذا الموضوع.

المهارات

المهارات المعرفية:

العمليات والاستراتيجيات المعرفية:

- التفسير
- التعلم الموائم
- حل المسائل

المهارات الشخصية:

الانفتاح الفكري:

- التعلم المستمر
- الاهتمام الفكري والفضول

بدء النشاط

في مجموعات صغيرة، قم بإنشاء ملصق لثلاث أفكار لبرنامج كمبيوتر يقوم بشيء تقوم به كل يوم دون استخدام أجهزة الكمبيوتر. وبالنسبة لكل فكرة، اقترح منصة يمكن تطويرها من أجلها (الهاتف المحمول، الويب، تطبيق سطح المكتب) واللغة التي يمكن تطويرها بها.

هدف التعلم (ب): تصميم برامج لتلبية متطلبات المستخدم

يبدأ كل برنامج بفكرة، "لحظة ضوء المصباح"، فكرة ربما يمكن لبرنامج ما أن يجعل حياتي أو وظيفتي أسهل. بالنسبة للعديد من المؤسسات، يأتي هذا من "العملاء" الداخليين أو الخارجيين للشركة. وبصفتنا مبرمجين للبرامج، فإننا نكتب برامج لحل المسائل وجعل مؤسساتنا أكثر كفاءة. تنطوي البرامج على غرض معين ومتطلبات تحتاج إلى التحول إلى حلول قابلة للاختبار والنشر. في هذا الموضوع، سوف نستكشف أنواع المتطلبات التي قد يتم توفيرها لك كمبرمج ونمنحك مجموعة المهارات اللازمة لإنتاج برنامج يفي بها.

(ب1) أدوات التصميم

تقييم متطلبات المستخدم

"متطلبات المستخدم" هي وصف يقدمه الفرد أو المؤسسة التي تطلب برنامجًا، لما هو مطلوب من البرنامج.

ستنص متطلبات المستخدم المثالية على "أحتاج إلى برنامج للقيام بشيء ما، لأن الطريقة الحالية تكلف المزيد من المال، وتستغرق المزيد من الوقت، وغير ممكنة، وخطيرة، وما إلى ذلك". لدينا "WHAT" (ماذا) من المستخدم - المشكلة التي نحتاج إلى حلها، و"WHY" (لماذا) - العاطفة وراء الطلب. تساعد عاطفة التغيير في تصميم أفضل حل لحل المشكلة.

خذ متطلبات المستخدم "السيئة" هذه بدون WHY:

"أريدك أن تصنع لي شيئًا بأربع أرجل". ماذا تفعل؟ طاولة؟ كرسي؟ كلب؟ حصان؟

دعنا نضيف "WHY" (لماذا) إلى المتطلبات:

"أريدك أن تصنع لي شيئًا بأربعة أرجل، لأنني أسقط باستمرار من على هذا الكرسي ذي الرجلين". ما الفرق الذي يحدثه "WHY" في المتطلبات!

سيركز باقي دليل التسليم هذا على تصميم وتطوير جزء من التطبيق الموضح أدناه. يتم تعريف وثائق التصميم للمشروع في مربعات.

دعونا نلقي نظرة على المتطلبات الواقعية من شركة هندسية.

تقوم خدمات التفتيش التابعة لشركة صالح بإجراء مسوحات مستقلة حول أعمال البناء في الأردن. إنهم يريدون جعل حضورهم على وسائل التواصل الاجتماعي أكثر نشاطًا وجلب المزيد من التخصيص. لديهم شخص واحد يعمل في مجال التسويق داخل المؤسسة، يدعى علي، وهم يجدون صعوبة في تسجيل الدخول إلى جميع مواقع التواصل الاجتماعي الفردية لنشر التحديث اليومي للمشاريع والأحداث المثيرة الجارية والقادمة. في بعض الأحيان ينسى علي نشر التحديث على جميع المواقع، وأصبحت مهمة عادية ومتكررة. لقد طلب منك علي، بصفتك مبرمجًا، تصميم وتطوير برنامج يسمح له بإنشاء منشور أساسي وإرساله إلى موقع الإنترانت الخاص بالشركة (نظام إدارة المحتوى المدعوم بقاعدة بيانات SQLite) وصفحات LinkedIn® وFacebook و®X. للحصول على تمويل لإكمال التطبيق الكامل، يريد منك إنشاء تطبيق نموذجي يتم نشره على الموقع الداخلي أولاً، قبل أن يطلب من الإدارة التوقيع على أعمال التطوير للاندماج في منصات التواصل الاجتماعي الكبيرة.

توثيق المتطلبات

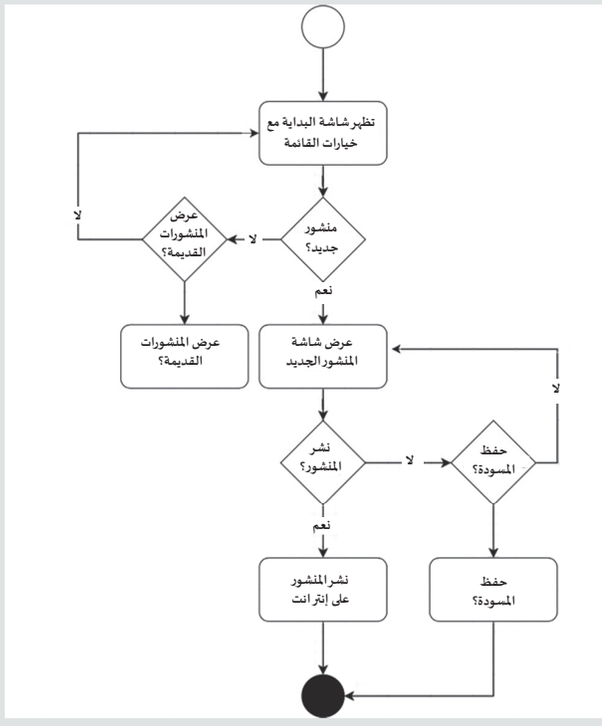
نحتاج إلى كتابة متطلبات المستخدم بتنسيق واضح وموجز. يحدد هذا المستند نطاق مشروعنا (يشار إليه أحيانًا باسم "النطاق" في فرق تطوير البرمجيات). يمنح هذا المستند المستخدم الفرصة لتحسين وتوضيح أي متطلبات ويعطي المبرمجين نقطة مرجعية أثناء المشروع. كما أنه مكتوب بمصطلحات غير مبرمجة حتى يتمكن مستخدمو الأعمال من فهم نطاق المستند و"الموافقة عليه".

دعونا نحدد متطلبات high level "المستوى العالي". عادةً ما تكون هذه المتطلبات هي WHAT وWHY لمتطلبات المستخدم.

> ***المتطلبات***
>
> كن قادرًا على نشر منشور واحد منسق على وسائل التواصل الاجتماعي على الإنترانت ولينكدإن وفيسبوك وإكس في إجراء واحد، نظرًا لأوجه القصور وعدم الدقة في إنشاء المحتوى يدويًا ونشره على كل منصة.

تدفق البرنامج

وفي إطار مساعدة علي والشركة على فهم ما سيفعله البرنامج وكيف "سيتدفق"، يمكننا إنشاء مخططات التدفق لتوضيح الخطوات والقرارات المتخذة داخل البرنامج. تحدد مخططات التدفق نقطة البداية للبرنامج، وتنتقل عبر المراحل الرئيسية للبرنامج وتطرح أسئلة يجاب عنها بـ"نعم" أو "لا" لنقاط القرار داخل البرنامج. نقاط القرار هذه هي المكان الذي يقوم فيه جهاز الكمبيوتر أو المستخدم بالاختيار. تستخدم مخططات التدفق *الأشكال* المعينة لتمثيل القرارات ولها مخرجان هما "نعم" و"لا" للقرارات المتخذة. تمثل المستطيلات المستديرة عملية داخل البرنامج وتمثل الدوائر الواضحة والفارغة بداية العملية ونهايتها.



الشكل 27.50 مخطط التدفق الذي يوضح تدفق البرنامج.

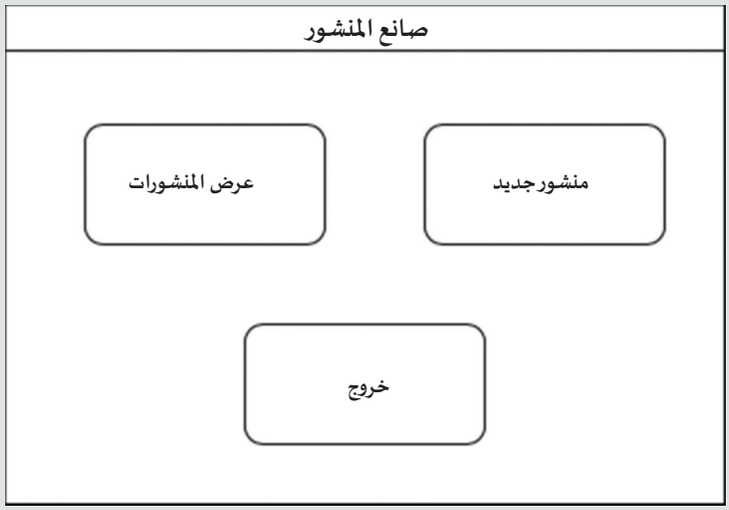
تصميمات واجهة المستخدم (UI)

تساعد تصميمات واجهة المستخدم قراء مستند النطاق على تصور المنتج النهائي. يمكنك إنتاج رسومات أولية وإطارات سلكية ولقطات شاشة مشروحة وصور ورسومات لتصوير تصميماتك للمستخدمين.

تقدم الرسومات الأولية أو الإطارات السلكية نظرة عامة موجزة عن واجهة المستخدم التي سيتم إنشاؤها. تعد لقطات الشاشة المشروحة مفيدة لتغييرات واجهة المستخدم على منتجات البرامج الحالية. يمكن إنشاء هذه اللقطات بواسطة الكمبيوتر باستخدام برامج رسم النظام مثل MS Paint أو برامج الرسوم البيانية التجارية مثل Microsoft Visio.

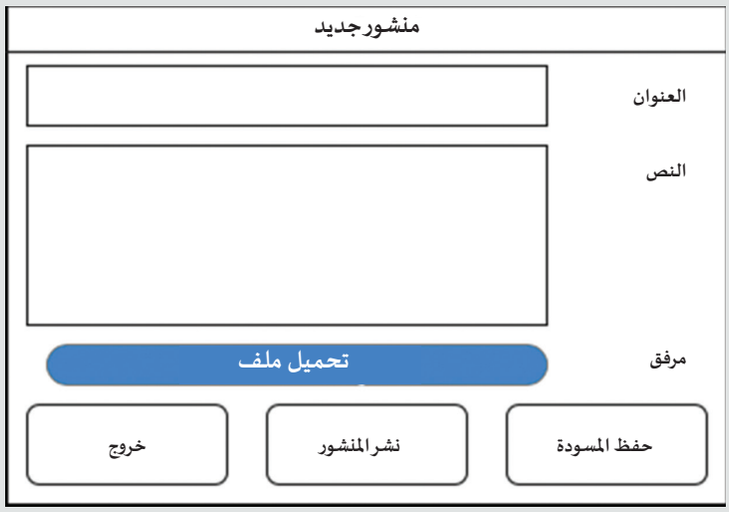
تصميمات الشاشة

الشاشة الرئيسية



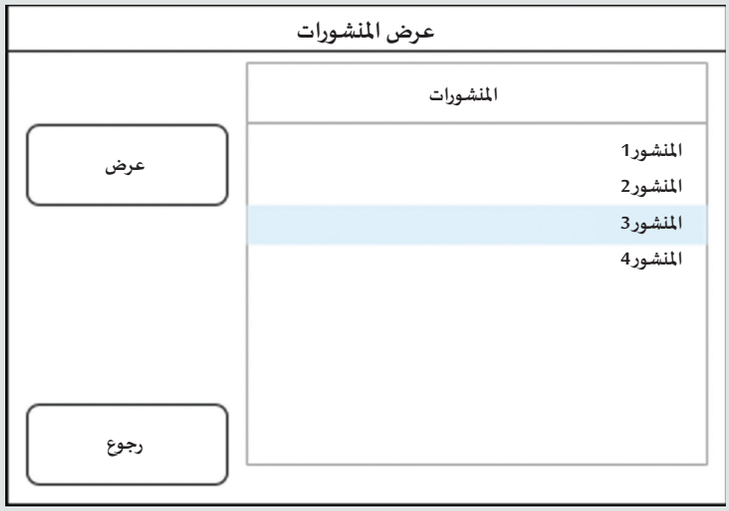
الشكل 27.51 واجهة مستخدم الشاشة الرئيسية.

شاشة النشر الجديد



الشكل 27.52 واجهة مستخدم النشر الجديد.

شاشة عرض المنشورات



الشكل 27.53 واجهة مستخدم عرض المنشورات.

(ب2) التخطيط للاختبار

أخيرًا، نوضح كيف سنختبر تطبيقنا. فهذا يمنح القارئ الثقة بأن البرنامج الذي تم إنتاجه سيكون خاليًا من الأخطاء ومناسبًا لأغراضه. وفي هذا القسم من المستند الخاص بنا، سنحدد الاختبارات التي سيجريها المطورون أثناء وبعد الانتهاء من مهمة البرمجة. وستكون بمثابة اختبارات إجرائية بسيطة، مثل الإجراءات التي يتم إجراؤها في Test Plan "خطة الاختبار"، وUnit Tests "اختبارات الوحدة"، وهي اختبارات آلية سيتم تشغيلها على البرنامج النهائي. تعمل Unit Tests (اختبارات الوحدة) على تعزيز اختبار الأتمتة وتقليل تكاليف ووقت الاختبار التقليدي.

خطط الاختبار والبرامج النصية

لاختبار برنامجنا النهائي، نحتاج إلى تحديد خطة اختبار وبرنامج نصي قبل بدء التطوير. مرة أخرى، هذه ضوابط وتعريفات إضافية لدعم تطوير البرنامج. تساعد البرامج النصية للاختبار المطورين على كتابة التعليمات البرمجية وفقًا للإجراءات التي سيتخذها المستخدم. تُعرف كتابة التعليمات البرمجية لاجتياز الاختبار في test plan (خطة الاختبار) بالتطوير القائم على الاختبار. على الرغم من أن منهجية التطوير هذه ليست مفصلة داخل هذه الوحدة، إلا أنها لا تزال طريقة رائعة لضمان تلبية البرنامج لمتطلباته عند تطويره.

يتم تحديد خطط الاختبار من خلال العنوان والهدف. من المفيد تحديد الغرض من خطة الاختبار لإعطاء سياق للاختبار. تتكون خطة الاختبار من سلسلة من البرامج النصية للاختبار. يتم تعريف برامج نصوص الاختبار هذه، مرة أخرى من خلال عنوان وهدف الاختبار الذي يتم إجراؤه، وسلسلة من الخطوات التي يجب اتباعها أثناء الاختبار. يحتوي كل اختبار على الحالة المتوقعة للبرنامج بعد الاختبار، والمعروفة باسم expected results "النتائج المتوقعة". يتبع ذلك حقل فارغ لactual results "النتائج الفعلية"، حيث يقوم المختبر بإدخال نتائج الاختبار، جنبًا إلى جنب مع النجاح أو الفشل. الاختبارات الفاشلة هي المكان الذي يؤدي فيه الاختبار إلى نتيجة غير متوقعة. ويتم توثيقها على أنها "أخطاء" حتى يقوم المطور بمراجعتها وإعادة تقديمها للاختبار.

ولإجراء الاختبار بكفاءة ودقة، بطريقة غير متحيزة، يجب ألا يقوم مطور دالة أو ميزة في برنامج باختبارها. في العديد من فرق تطوير البرامج الصغيرة، يقوم المطورون باختبار التعليمات البرمجية لبعضهم البعض ضمن إصدار تطوير البرنامج. هذه "بنية" صغيرة من التطبيق قابلة للاختبار.

خطة الاختبار والبرنامج النصي

الاختبار 1

- المطلب: كن قادرًا على الاختيار بين New Post (منشور جديد) أو View Posts (عرض المنشورات) أو Quit (الإنهاء) على الشاشة الأولى.
- الخطوات: قم بتحميل البرنامج، واعرض الشاشة الأولى، وراجع خيارات New Post (منشور جديد)، أو View Posts (عرض المنشورات) أو Quit (الإنهاء).
- النتائج المتوقعة: عند تحميل البرنامج، يجب أن تكون هناك ثلاثة خيارات قائمة لـNew Post (منشور جديد)، وView Posts (عرض المنشورات) وQuit (الإنهاء)
- النتائج الفعلية: يتم إكمالها أثناء الاختبار، وتحتوي على تفاصيل نتائج الاختبار. النتائج المتوقعة تؤدي إلى Pass "النجاح"، والنتائج غير المتوقعة تؤدي إلى Fail "الفشل".

...

اختبارات الوحدة

اختبارات الوحدة عبارة عن اختبارات مشفرة داخل البرنامج. ففي Visual Studio، يتم تنفيذ هذا الاختبار في Test Project' "اختبار المشروع" الذي يشير إلى البرنامج. تمثل اختبارات الوحدة طرقًا لاختبار وحدة معينة في البرنامج؛ أو طريقة، أو إجراء، أو دالة محددة. كان اختبار الوحدة موجودًا منذ أوائل الخمسينيات وهو جزء مهم من البرمجة. حيث إنه يساعد على تحديد العيوب في تعليمات البرمجة بشكل مبكر، مما يؤدي إلى عدد أقل من الأخطاء أثناء مرحلة الاختبار العامة.

اختبارات الوحدة

اختبار وحدة واحد محتمل للتطبيق. في هذه المرحلة من تصميم البرنامج، سيفشل هذا الاختبار بسبب عدم وجود تعليمات برمجية وظيفية لنشر منشور.

```
[TestClass]
0 references
public class UnitTests{
    [TestMethod]
    0 references
    public void TestNewPostSave(){
        var post = new IntranetPost();
        post.Title = "Unit Test Post";
        post.Body = "This is a test post from unit testing.";
        post.Attachment = "C:/testfiles/test.png";
        bool result = post.PostToIntranet();
        Assert.IsTrue(result);
    }
}
```

الشكل 27.54 مثال على التعليمات البرمجية لاختبار الوحدة.

اختبارات ومراجعات المطور

يتم إجراء اختبارات ومراجعات المطورين أثناء وبعد فترة وجيزة من تطوير جزء من التعليمات البرمجية. يعمل المطورون معًا على "توقف" البرنامج قبل أن يصل إلى أي نوع رسمي من الاختبارات مثل اختبار الوحدة أو خطط الاختبار. تتضمن اختبارات المطور ما يلي:

- اختبار إتلافي، والبحث عن طرق لوقف التعليمات البرمجية الخاصة بك، مثل إدخال معلومات غير صحيحة عن قصد، أو النقر المزدوج أو الثلاثي على عناصر واجهة المستخدم لمعرفة ما إذا كان يتم استدعاء إجراءات متعددة.
- خطوات الاستنساخ أو القبول، حيث يتبع المطورون الخطوات في مهمة عملهم للتأكد من أن التعليمات البرمجية الخاصة بهم تلبي متطلبات النطاق والمهمة التي عملوا عليها. تظهر هذه الخطوات عادةً باسم Acceptance Criteria "معايير القبول" أو Steps to Reproduce "خطوات إعادة الإنتاج" على عناصر العمل.
- مراجعات الأقران، حيث يراجع المطورون التعليمات البرمجية لمطور آخر. تتألف هذه المراجعات من فحص مرئي للتعليمات البرمجية بحثًا عن أي أخطاء نحوية واضحة أو كتل تعليمات برمجية غير فعالة. ففي فرق البرمجة الصناعية، يعمل المطورون عادةً على branches "فروع" من البرنامج الرئيسي، والتي تسمى عادةً "dev" أو "main". وعند اكتمال جزء من العمل، يقوم مطور العمل بتقديم pull request "طلب سحب" يطلب من المجموعة دمج هذا الجزء الجديد من العمل في البرنامج الرئيسي. تعد هذه العملية ضرورية لتمكين العديد من المطورين من العمل على مشروع واحد معًا دون إنتاج تعليمات برمجية متعارضة وهي طريقة مهمة لضمان مراجعة كل جزء من التعليمات البرمجية المكتوبة لبرنامج ما من قبل الأقران.

في مشاريع البرمجة الجديدة، يكون المطور مسؤولاً عن creating "إنشاء" البرنامج الأولي. وعادة ما يكون هذا هو إنشاء المشروع في بيئة التطوير المتكاملة، وتعريف نقطة الدخول الرئيسية. بمجرد اكتمال نقطة تشغيل البرنامج الرئيسية، حينئذٍ تصبح متاحة للعمل عليها من قبل مطور واحد أو أكثر. وفيما يتعلق بهذه الوحدة، من المتوقع أن يتم تطوير البرنامج من قبل مطور واحد ولن يتضمن العمل الجماعي. لا يدخل استخدام التحكم في المصدر للبرمجة متعددة المطورين في نطاق هذه الوحدة.

المهارات

المهارات المعرفية: العمليات والاستراتيجيات المعرفية

- التفسير
- التعلم الموائم
- حل المسائل

الإبداع:

- الإبداع/الابتكار

المهارات الشخصية: الانفتاح الفكري:

- التعلم المستمر
- الاهتمام الفكري والفضول

أخلاقيات العمل/الضمير:

- المبادرة
- التوجيه الذاتي

مهارات التواصل الشخصي: العمل الجماعي والتعاون:

- الاتصالات
- التعاون
- التعاطف/تحديد المنظور

القيادة:

- العرض الذاتي

ممارسة التقييم هدف التعلم (ب)

هدف التعلم (ب): تصميم برامج لتلبية متطلبات المستخدم

لقد طلب منك مصنع محلي تصميم برنامجًا لكشك صغير بشاشة تعمل باللمس موجود في مكتب الاستقبال. ويرغبون في أن يقوم جميع الزوار بالتسجيل في هذا الكشك قبل زيارة المصنع. يحتوي الكشك على كاميرا وشاشة تعمل باللمس واتصال بالشبكة وطابعة حرارية مع ورق ذاتي اللصق. إنهم يرغبون في تقديم تصميم البرنامج قبل الالتزام بالتطوير.

صمم برنامجًا لتلبية متطلبات المستخدم التالية:

- السماح لزوار مؤسسة هندسية باستخدام كشك الزوار الإلكتروني لتسجيل حضورهم والموافقة على تعليمات السلامة عند زيارة موقع الشركة. يجب أن يقوم برنامج الكشك بما يلي:
  - جمع الاسم واسم الشركة وتاريخ ووقت الزيارة وتفاصيل الاتصال في حالات الطوارئ حول الزائر
  - التقاط صورة للزائر
  - تخزين تفاصيل الزائر في قاعدة بيانات محلية
  - طباعة شارة الزيارة وإعطائها للزائر لارتدائها أثناء زيارة الموقع
  - التوافق مع سياسات الصحة والسلامة الخاصة بالشركة.

قم بتقييم متطلبات المستخدم وتحديد أدوات البرامج والأجهزة المطلوبة لتنفيذ مرحلة تطوير المشروع. وقم بإنشاء مخططات التدفق وتصميمات واجهة المستخدم وأوامر التعليمات للتطبيق. قم بإنشاء خطط اختبار وبرامج نصية للتحقق من صحة البرنامج في ضوء متطلبات المستخدم والمسائل غير المتوقعة.

سيكون الدليل على ممارسة التقييم هذه عبارة عن مجموعة من مستندات التصميم لاستخدامها في تطوير البرنامج.

هدف التعلم (ج): تطوير برامج لتلبية متطلبات المستخدم

حان الوقت الآن لوضع محتوى هذه الوحدة موضع التنفيذ. في هذا الموضوع، سنأخذ مستند النطاق الخاص بنا من الموضوع السابق ونقسمه إلى مهام تطوير دقيقة محددة. وسنستخدم هذه المهام لتحديد اللغة والنظام الأساسي الأنسب للتطوير، حتى نتمكن من إنتاج تطبيق يعمل بكامل طاقته يلبي متطلبات المستخدم ويجتاز جميع أنواع الاختبارات.

(ج1) تخطيط المهام

من مستند النطاق، نحتاج إلى "تقسيم" متطلباتنا عالية المستوى إلى متطلبات أصغر وأكثر تحديدًا. تحدد هذه المتطلبات الإضافية للمبرمجين. ويطلق عليها اسم المتطلبات الفنية وتشكل أساس "مهامنا" للمطورين للعمل عليها. وبالنسبة لمشروعنا كما هو محدد في الموضوع السابق، فيما يلي المتطلبات الفنية:

بدء النشاط

في مجموعات، حدد المنصات الأكثر ملاءمة (شبكة الويب العالمية، الإنترانت، الهاتف الذكي، التلفزيون الذكي، وحدة التحكم في الألعاب، كمبيوتر سطح المكتب) لتطوير البرامج التالية من أجل:

- تطبيق لخدمة بث تلفزيوني جديدة
- لعبة محاكاة بناء مدينة
- بائع تجزئة لموقع ويب خاص بالتجارة الإلكترونية.

المتطلبات الفنية

- قم بإنشاء واجهة مستخدم تسمح للمستخدم بإدخال العنوان ونص المنشور ومرفقات الصور الاختيارية.
- قم بتطوير التكامل في الإنترانت:
  - أضف جدولًا إلى قاعدة بيانات إنترانت لمنشورات الوسائط الاجتماعية، يسمى Posts "المنشورات" مع الأعمدة ID (int) و Title (string) و Body (string) و DateCreated (datetime) و CreatedBy (string) و Published (bool):
    - ID (المعرف) - عدد صحيح - مفتاح أساسي متزايد تلقائيًا - تم إنشاء النظام
    - Title (العنوان) - السلسلة - عنوان المنشور - تم إنشاء مستخدم
    - Body (النص) - السلسلة - نص المنشور - تم إنشاء مستخدم
    - DateCreated (تاريخ الإنشاء) - التاريخ والوقت - تاريخ ووقت المنشور - تم إنشاء النظام
    - createdBy (إنشاء بواسطة) - السلسلة - اسم المستخدم - تم إنشاء النظام
    - Published (تم النشر) - قيمة منطقية - صواب أو خطأ إذا تم نشر المنشور أم لا.
  - أضف جدولًا إلى قاعدة بيانات إنترانت لـPost Attachments (مرفقات المنشورات)، يسمى "postAttachments" مع أعمدة ID (int) و postID (int) و Filename (string):
    - ID (المعرف) - عدد صحيح - مفتاح أساسي متزايد تلقائيًا
    - postID (معرف المنشور) - عدد صحيح - رابط إلى جدول المنشورات، كمفتاح خارجي
    - Filename (اسم الملف) - السلسلة - المسار إلى ملف المرفقات.
  - أضف طريقة الحفظ لمتجر بيانات SQLite لتخزين Title (العنوان) وBody (نص المنشور) والمرفقات إلى الجداول الجديدة.

- أضف زرًا إلى واجهة المستخدم لحفظ المنشور وزرًا لإرسال المنشور إلى المواقع المستهدفة:
- قم باستدعاء طريقة للحفظ في مخزن بيانات SQLite للإنترانت
- اترك العناصر النائبة لمواقع التواصل الاجتماعي الأخرى في أعمال التطوير المستقبلية
- في حالة الضغط على save (حفظ)، احفظ البيانات في قاعدة البيانات مع تعيين كلمة published "منشور" على false (خطأ)
- في حالة الضغط على النشر، احفظ البيانات في قاعدة البيانات مع تعيين كلمة published "منشور" على true (صواب).

نحن نتطلع إلى تحديد متطلبات المستخدم في الحد الأدنى لعدد المهام التي تلبي متطلبات المستوى العالي. يُعرف هذا الإصدار من المنتج باسم "MVP". إنه الحد الأدنى من المنتجات القابلة للتطبيق التي يمكن إنتاجها لتلبية متطلبات المستخدم.

المهام المذكورة أعلاه ليست بعد بالتنسيق الذي يتوقعه المطور. كما أن المهام الخاصة بالمطورين عبارة عن كتل قائمة بذاتها تحتوي على التفاصيل التي يحتاجونها لإكمال المهام. هناك العديد من مواقع الويب التي تقدم تخطيط المهام وإدارتها عبر الإنترنت لفرق البرامج وتستخدم نظام card "البطاقة" لتتبع مراحل المهمة ومراقبتها أثناء التطوير.

الجدول 3-27 لوحة المهام مع حالات العمل لكل عمود

الأحداث	ما ينبغي فعله	قيد التقدم	الاختبار	المنجزات
المهمة 9	المهمة 7	المهمة 4	المهمة 2	المهمة 1
المهمة 10	المهمة 8	المهمة 5	المهمة 3	
المهمة 11		المهمة 6		

Card (البطاقة)

تتكون card "البطاقة" من المعلومات الأساسية التي يحتاجها المطور لكتابة التعليمات البرمجية.

وعادةً ما تتكون هذه البطاقة من الأجزاء التالية من المعلومات:

العنوان/الملخص

اسم موجز لتحديد المهمة، على سبيل المثال، إنشاء واجهة مستخدم لPost Builder (منشئ المنشور).

الوصف

وصف تفصيلي للمهمة، يتم تعبئته عادةً من مستند النطاق. ويوضح بالتفصيل المعلومات التي يحتاجها المطور لتطوير الميزة بدقة أو إصلاح الخطأ، على سبيل المثال، "إنشاء واجهة المستخدم لعنصر منشئ المنشورات في التطبيق". أضف تسميات وحقول نصية لTitle (العنوان) وPost Body (نص المنشور). يلزم وجود عنصر تحكم في تحميل الملفات للسماح للمستخدم بإضافة مرفقات إلى المنشور. ويجب تخزين المعلومات التي تم إدخالها في كائن داخل البرنامج جاهز للحفظ والنشر. يعتبر حقلا العنوان ونص المنشور إلزاميان، لكن المرفقات اختيارية. تأكد من أن Save Draft "حفظ المسودة" وPublish Post "نشر المنشور" وClear Post "مسح المنشور" قد حددوا أحداثًا وأن الطرق ذات الصلة قد تم إنشاؤها وربطها بالأحداث."

المعين له

المطور المعين للمهمة.

المرحلة/الحالة

حالة المهمة، على سبيل المثال، المرحلة الأولى هي "ما يجب القيام به".

الوقت المقدر

تقدير الوقت (الدقائق والساعات والأيام والأسابيع) لإكمال نسبة تطوير المهمة بشكل كامل. لا يشمل ذلك أوقات الاختبار أو المراجعة، على سبيل المثال، 4h (4 ساعات). من المهم أن تكون الأوقات المقدرة واقعية، لضمان بقاء المشروع ضمن الموازنة والجداول الزمنية. تؤثر الأوقات دون التقدير المطلوب أو المبالغ في تقديرها على الموارد المتاحة للمشروع ويمكن أن يكون لها آثار ضارة.

الإصدار

إصدار البرنامج الذي سيتم تضمين هذه المهمة فيه، على سبيل المثال، سيكون التطبيق الجديد هو الإصدار 1.0.

خطوات إعادة الإنتاج (الخطأ) أو معايير القبول (الميزة)

خطوات هذه المهمة من خطة الاختبار ومستند النطاق التي يجب على المطور اتباعها عند تطوير التعليمات البرمجية واختبارها ذاتيًا في هذه المهمة، على سبيل المثال، تشغيل البرنامج، وإدخال عنوان المنشور باسم Test Title "عنوان الاختبار"، وإدخال نص المنشور باسم "هذا منشور اختبار من برنامج النشر". هذا "اختبار مطور"، أضف مرفق اختبار، اضغط على Post (منشور).

التعليقات

هذا هو المكان الذي يقوم فيه المطور بتحديث قائد المشروع أو المطورين الآخرين بحالة المهمة. هذه ميزة أساسية في العديد من الإعدادات الصناعية حيث تضمن المهام التي تم التعليق عليها الجودة داخل فرق التطوير، على سبيل المثال، "تم إنشاء واجهة مستخدم في Visual Studio" وحقول النص المضافة والتسميات لعناصر Title (العنوان) وBody (النص). البحث عن كيفية إضافة تحميلات الملفات إلى التطبيق لإضافة ميزة المرفقات. '.

مراحل التطوير

يتم تحديد مراحل التطوير من قبل المؤسسة والفريق الذي يقوم بأعمال التطوير، ولكنها تتكون عادةً من المراحل التالية:

ما ينبغي فعله

هذه مهمة مكلف بها تنتظر العمل عليها.

قيد التقدم

يجري العمل على المهمة بنشاط.

For review (للمراجعة)

هذه مهمة مكتملة، في انتظار مراجعة الأقران. إذا نجحت المراجعة فإنها تنتقل إلى In Testing (قيد الاختبار). وإذا فشلت، فإنها تعود إلى In Progress (قيد التقدم) مع ذكر أسباب ذلك.

In testing (قيد الاختبار)

تمت مراجعة هذه المهمة وتنتظر اختبارها. إذا نجحت المهمة في الاختبار، فإنها تنتقل إلى Complete "الاكتمال"، وإذا فشلت فإنها تعود إلى In Progress "قيد التقدم" مع ذكر أسباب ذلك.

Complete (الاكتمال)

تم الانتهاء من هذه المهمة وإنهائها.

النشاط

1. قم بإنشاء مهام المطور لجميع المتطلبات الموثقة في هذا المشروع.
2. وقم بإعداد تقدير لعدد الساعات المطلوبة لتطوير هذا البرنامج.
3. قم بتبديل قائمة المهام الخاصة بك مع أحد الزملاء واطلب منهم مراجعة المهام الخاصة بك في ضوء مستند النطاق.

هل قمت بتجميع كل متطلبات المستخدم؟ إن ضمان تلبية مهامك لمتطلبات المستخدم في هذه المرحلة المبكرة يقلل من احتمالية الاختبارات الفاشلة في مرحلة الاختبار ويحسن الجودة الشاملة لعملية التطوير التي تقوم بها.

المصطلحات الرئيسية

خارطة الطريق: جدول تخطيط رشيق لدعم الأهداف طويلة الأجل والمستقبلية للمشروع.

تلبي قائمة المهام الخاصة بنا متطلباتنا الأولية، لكن علي يريد معرفة خطط عمليات الدمج الأخرى في فيسبوك ولينكدإن وإكس. في هذه المرحلة من المشروع، لا نريد قضاء الكثير من الوقت في التخطيط للإصدارات المستقبلية من البرنامج حتى يتم تطوير الإصدار الأول أو الحالي وإصداره. نحتاج أن نوضح لعلي كيف سنقوم بتطوير البرنامج في المستقبل، ولكن دون الخوض في الكثير من التفاصيل من وجهة نظر فنية. نحن نفعل ذلك باستخدام خارطة الطريق.



الشكل 27.55 خارطة طريق مع العلامات أو المعالم الرئيسية.

يمكن تقديم خارطة الطريق في تقرير أو جدول بيانات أو حتى لوحة خريطة طريق رقمية مثل لوحة Trello. تحتوي جميع خرائط الطريق على نفس المفهوم الأساسي: ما هي الميزة الجديدة ومتى يتم إصدارها. هذه هي الأجزاء الرئيسية من المعلومات التي يطلبها أصحاب المصلحة في المشروع.

دعونا نخطط لخارطة الطريق الخاصة بنا:

الجدول 4-27 خارطة الطريق

الميزة	الملخص	الإصدار	تاريخ الإصدار
التكامل مع فيسبوك	التكامل في فيسبوك باستخدام واجهات برمجة التطبيقات	1.1	نهاية الربع الثاني (Q2)
تكامل X	التكامل في إكس باستخدام واجهات برمجة التطبيقات	1.2	نهاية الربع الثالث
تكامل لينكد إن	التكامل في لينكدإن باستخدام واجهات برمجة التطبيقات	1.3	نهاية الربع الرابع

المصطلحات الرئيسية

الربع: فترة زمنية ضمن السنة التشغيلية للمؤسسة باستخدام تعريف "أرباع السنة"، على سبيل المثال، يغطي الربع الأول أو Q1 أشهر يناير وفبراير ومارس. ستعطي "نهاية الربع الثاني" وقتًا تقديريًا للتسليم حتى وقت ما خلال شهر يونيو.

خرائط الطريق هي جداول مرنة لأعمال التطوير المستقبلية. وتعد أداة مفيدة للمؤسسة أو المستخدم لإعادة ترتيب أولويات أعمال التطوير المستقبلية.

النشاط

بعد مشروع نشر التحديثات الاجتماعية على موقع الإنترانت، طلبت الشركة منك تطوير نظام إدارة علاقات العملاء (CRM) لمساعدة الشركة على النمو والاندماج فقط في لينكدإن لأنهم يشعرون أن الشبكات الاجتماعية ليست مفيدة. وقالوا:

"نحن بحاجة ماسة إلى نظام إدارة علاقات العملاء الخاص بنا بحلول نهاية الربع الثاني"

"نحن بحاجة إلى التركيز على مبدأ "معاملات الشركات فيما بينها". ومن الأهمية بمكان أن يتم تطويره بحلول نهاية الربع الثالث."

"نريد أن نعرف العملاء الذين لم يطلبوا منذ فترة حتى نتمكن من المتابعة معهم."

بعض المتطلبات أساسية وحاسمة، ويجب إكمالها وفقًا للجداول الزمنية المذكورة. تستخدم هذه الكلمات الرئيسية "حرجة" أو "ماسة" أو تشير إلى إطار زمني. سيتم تطوير متطلبات أخرى في المستقبل.

قم بتحديث خارطة الطريق للمشروع لجدولة هذه المتطلبات الجديدة مع تواريخ التسليم والإصدارات المقدرة.

باختصار، يعد تخطيط المهام ورسم خرائط الطرق جزءًا أساسيًا من المشروع. إذ إنه يساعد في تحديد الحد الأدنى من المتطلبات اللازمة لإنتاج حل. يساعد الحفاظ على عمل المطور عند الحد الأدنى في المراحل الأولى من المشاريع الكبيرة على ضمان تسليم المشروع في الوقت المحدد

(ج2) التطوير

اختيار اللغة

بعد اكتمال التصميمات، تم الاتفاق على خارطة الطريق (في الوقت الحالي) وتم إعطاؤك "الضوء الأخضر"، للإشارة إلى أنه قد حان الوقت لبدء البرمجة.

كيف تبدأ؟ ما اللغة التي تختارها؟

يجب اختيار اللغة والمنصة المناسبتين لتناسب المشروع الذي يتم العمل عليه. يمكن أن يساعد النظر إلى المتطلبات الأولية وخارطة الطريق في تحديد كيفية بناء البرنامج. ثمة العديد من العوامل التي تؤثر على اختيار لغة البرمجة. على سبيل المثال:

- لقد طُلب منك إنشاء موقع ويب يقوم العديد من المستخدمين بتسجيل الدخول إليه من أي مكان - الهاتف أو الكمبيوتر الشخصي. تحتاج إلى استهداف بعض لغات برمجة الويب لهذا المشروع، ومن المحتمل أن تستخدم ASP.NET C# أو VB.NET أو MVC C # أو VB.NET أو PHP أو JavaScript على سبيل المثال لا الحصر.



الشكل 27.56 تصميم موقع ويب على جهاز الكمبيوتر.

- لقد طُلب منك إنتاج تطبيق يعمل على كشك بشاشة تعمل باللمس في مجمع تسوق يبيع بطاقات الهدايا لمتاجره. ويحتاج التطبيق إلى الاتصال بقارئ بطاقات الائتمان وقارئ العملات المعدنية والورقية وطابعة للإيصالات. كما يجب أن يكون قادرًا على العمل بدون الإنترنت إذا كانت هناك مشكلات في الاتصال. تحتاج إلى استهداف بعض لغات برمجة سطح المكتب لهذا المشروع ومن المحتمل أن تستخدم WPF أو WinForms مع C# أو Java أو Python أو C لإنتاج هذا البرنامج على سبيل المثال لا الحصر.

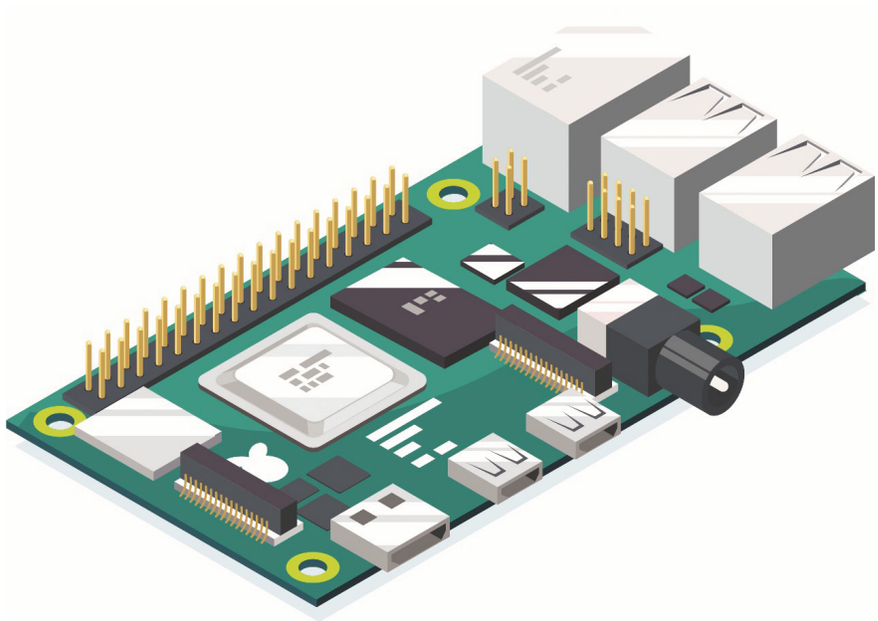


الشكل 27.57 شخص يستخدم كشك بشاشة تعمل باللمس.

- لقد طُلب منك إنتاج برنامج لموزع مياه أوتوماتيكي لحقل مزارع. سيتم تشغيل البرنامج على جهاز **إنترنت الأشياء** الصغير المتصل بلوحة شمسية للطاقة وعامل تصميم الكمبيوتر الصغير، مثل RaspberryPi®. تحتاج إلى استهداف بعض لغات برمجة إنترنت الأشياء لهذا المشروع ومن المحتمل أن تستخدم .NET Core أو Python أو Java أو واحدة من العديد من اللغات الأخرى المتاحة.

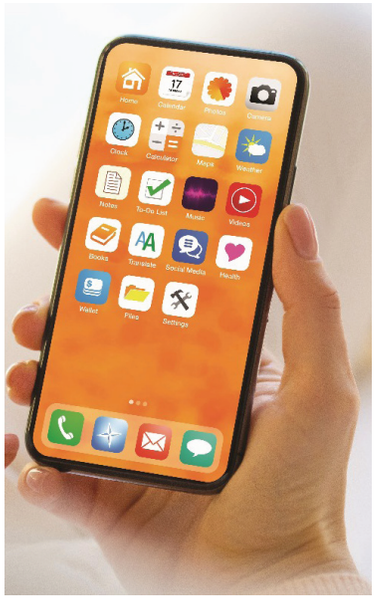
المصطلحات الرئيسية

إنترنت الأشياء: "IoT"، عبارة عن تقنية تسمح لنا بإضافة جهاز إلى كائن خامل.



الشكل 27.58 كمبيوتر ذو عامل تصميم صغير.

- لقد طُلب منك إنتاج تطبيق لهاتف محمول يعمل بنظام iOS يتنبأ بموعد اكتمال القمر التالي. ونظرًا لأن هذا تطبيق لنظام تشغيل خاص بالهاتف المحمول، فمن المحتمل أن تستهدف Cocoa أو Swift لأن هذه التطبيقات مخصصة لتطوير iOS.



الشكل 27.59 هاتف ذكي.

يعود اختيار لغة البرمجة إلى إيجاد الحل الأسهل والأكثر كفاءة لحل المسألة. هناك فائدة قليلة لاختيار لغة برمجة ليس لديك خبرة أو معرفة بها لحل المسألة.

ممارسات الترميز الجيدة

تمت تغطية بنيات الترميز في الموضوعات السابقة لهذه الوحدة. تعد التعليمات البرمجية النظيفة والدقيقة هي المفتاح لإنشاء برنامج يعمل بكامل طاقته و"يجتاز الاختبار". يجب أن يهدف المبرمجون إلى كتابة تعليمات برمجية أنيقة ومرتبة وقابلة للقراءة من قبل مطورين آخرين. يعد العمل ضمن فريق من المطورين طريقة رائعة لتطوير المهارات الشخصية والاجتماعية والتقنية. ولم يكن ذلك ممكناً إلا إذا كانت هناك طرق اتصال واضحة بين المطورين. وتعد التعليقات والتعليمات البرمجية المشروحة هي مفتاح الحفاظ على هذه الاتصالات.

عند تعلم البرمجة، غالبًا ما يُنظر إلى التعليقات على أنها "غير مجدية" أو "واضحة جدًا" ويتم تجاهلها. لا يتم تضمين التعليقات والتعليقات التوضيحية في البرنامج النهائي القابل للتنفيذ ويمكن استخدامها للتذكير بكتل التعليمات البرمجية وتوجيهها وتنظيمها. كما يمكن استخدامها لشرح الغرض أو الطرق والمعلمات، أو استخدامها ببساطة داخل كتلة التعليمات البرمجية للتركيز على المطور.

خذ المثال أدناه للتعليق على طريقة كمطور قبل كتابة أي منطق برمجة:

```
//Gets the post from the database for the given ID
0 references
private static Post GetPostForPostID (int PostID)
{
        //set up connection to SQLLite database

        //create query to get post body for ID

        //build Post object with results from SQLLite result
```

الشكل 27.60 استخدام التعليقات في تصميم التعليمات البرمجية.

يساعد كل سطر تعليق أخضر على تقسيم الكتلة الكبيرة والمعقدة من التعليمات البرمجية إلى أجزاء يمكن التحكم فيها.

(ج3) الاختبار

يعد الاختبار مرحلة حاسمة من تطوير المشروع ويجب إجراؤه أثناء كتابة التعليمات البرمجية وبعدها. إذ يتحقق الاختبار بشكل أساسي من قدرة البرنامج على تلبية متطلباته. تتراوح الأدوار المسؤولة عن الاختبار من المبرمجين وأصحاب المصلحة ومختبري ضمان الجودة إلى المستخدمين النهائيين أنفسهم.

يمكن أن يؤدي كل دور مختلف يتم اختباره أيضًا إلى نتائج مختلفة. يمكن اعتبار نتائج اختبار المطور "اختبارًا متحيزًا". يعرف المبرمجون كيفية "اجتياز الاختبار". لقد كتبوا التعليمات البرمجية وسيختبرون مدى معرفتهم بعملها. يمكن اعتبار النتائج من أصحاب المصلحة ومختبري ضمان الجودة والمستخدمين "اختبارًا غير متحيز" وستختبر تصورهم لطبيعة علم البرنامج بناءً على المتطلبات.

ثمة العديد من أنواع الاختبارات المختلفة لدعم هذا التحقق ويمكن استخدام كل نوع من الاختبارات لأغراض مختلفة للتوصل إلى نتائج مختلفة:

اختبار الوظائف

اختبار الوظائف يختبر البرنامج في ضوء متطلبات المستخدم الوظيفية. وعادةً ما يتم تعريف متطلبات المستخدم هذه على أنها اختبارات ضمن خطة اختبار وتُعرف باسم "أحداث المستخدم". وتوضح على وجه التحديد ما سيفعله المستخدم وسبب ذلك في البرنامج. هذه هي المتطلبات التي تشير إلى أنه "يجب على البرنامج القيام بهذا الإجراء والسماح لي بالقيام بشيء آخر". ستتضمن خطة الاختبار الخاصة بك، الإجراءات الوظيفية داخل البرنامج والتي يجب اختبارها أيضًا. تحتوي البرامج النصية للاختبار هذه على إجراءات مثل "تسجيل الدخول" أو "إضافة عنصر إلى سلة التسوق". ولإجراء الاختبار الوظيفي، يجب على الشخص المسؤول عن الاختبار الالتزام بالإرشادات التالية:

1. فهم متطلبات المستخدم
2. تحديد بيانات الإدخال أو الاختبار لاستخدامها أثناء الاختبار
3. تقديم النتائج المتوقعة بناءً على بيانات الإدخال والمتطلبات المحددة
4. إجراء الاختبارات
5. مقارنة النتائج الفعلية والمتوقعة من الاختبار.

عادةً ما يكون اختبار الوظائف هو النوع الأول من الاختبارات التي يتم إجراؤها ويمكن إجراؤه كجزء من اختبار الوحدة أو اختبار السلامة أو اختبار قبول المستخدم.

اختبار إمكانية الاستخدام

اختبار إمكانية الاستخدام، المعروف أيضًا باسم اختبار تجربة المستخدم (UX)، هو مقياس لمدى سهولة استخدام واجهة مستخدم البرنامج. وعادة لم يتم إجراء اختبار إمكانية الاستخدام من قبل الأطراف المشاركة في تطوير البرنامج. ويجيب اختبار إمكانية الاستخدام عن الأسئلة التالية للبرنامج:

الفاعلية

ما مدى فاعلية البرنامج؟ هل من السهل تعلم الاستخدام؟ هل هي مفيدة وتضيف قيمة إلى قاعدة المستخدمين؟ هل الألوان والمحتوى والأيقونات والصور تسر العين؟

الكفاءة

هل يتطلب تنفيذ إجراء ما عدد نقرات منخفض؟ هل التنقل دقيق؟ هل تتبع الشاشات والواجهات المطابقة؟ هل يمكنك البحث عن المعلومات في البرنامج بسهولة؟

الدقة

هل المعلومات دقيقة ومحدثة؟ هل هناك أي روابط معطلة أو أزرار غير قابلة للنقر فوقها على سبيل المثال؟

سهولة الاستخدام

هل الواجهة لا تحتاج إلى شرح؟ هل يوجد دليل مساعدة مضمن؟

كيف يعمل اختبار إمكانية الاستخدام؟

هذا أسلوب مختلف للاختبار قد تكون على دراية به. إنه مثل المنتدى حيث تقدم مجموعة من المستخدمين تعليقات حول كيفية تفاعلهم مع البرنامج.

فيما يلي خطوات إجراء اختبار إمكانية الاستخدام:

التخطيط

يتم تحديد أهداف الاختبار والخصائص الديموغرافية للمختبرين المطلوبين وأدوارهم ومسؤولياتهم. كما يتم أيضًا تحديد النتيجة المتوقعة من الاختبار، وهي تنسيق تقرير المخرجات.

التوظيف

يتم العثور على المختبرين الذين يتطابقون مع الخصائص الديموغرافية المستهدفة في هذه المرحلة، مثل العمر والنوع والمهنة وما إلى ذلك.

اختبار إمكانية الاستخدام

يتم تنفيذ اختبارات إمكانية الاستخدام في هذه المرحلة.

تحليل البيانات

يتم تحليل نتائج الاختبار الذي تم تنفيذه في المرحلة السابقة لتقديم توصيات قابلة للتنفيذ لتحسين المنتج.

إعداد التقارير

تتم مشاركة نتائج اختبار إمكانية الاستخدام مع أصحاب المصلحة في المنتج.

اختبار التراجع

يتم إجراء اختبار التراجع للتأكد من أن تغيير ميزة جديدة أو تغيير التعليمات البرمجية لم يؤثر سلبًا على وظائف البرنامج الحالية. كما يضمن أن التعليمات البرمجية القديمة لا تزال تعمل بغض النظر عن التغييرات الجديدة.

يمكن إجراء اختبار التراجع عندما:

- يتم إضافة وظيفة جديدة
- يتم تغيير المتطلبات
- يتم إصلاح الخطأ
- يتم إصلاح الأداء
- تكامل الأنظمة الخارجية الأخرى.

تتمثل الطريقة الأكثر شيوعًا لاختبار التراجع في "إعادة اختبار الكل" (Retest All). ويتضمن ذلك تنفيذ كل حالة اختبار سابقة مكتوبة للمنتج. يقوم المختبر باختبار كل شيء لضمان استمرار اجتياز جميع الاختبارات. يتبع اختبار التراجع بنية مشابهة لاختبار الوظائف حيث يتم استخدام المتطلبات لإنشاء مجموعة من الحالات للاختبار والإبلاغ عنها.

الاختبار الإتلافي

الاختبار الإتلافي هو استخدام غير مكتوب و"متلف" لبرنامج. ويساعد على فهم كيفية أداء البرامج عند استخدامها بشكل غير صحيح والتحقق من جودتها. في الاختبار الإتلافي، يقوم المختبر عادةً باختبار:

- أن البرنامج لا يعالج أو يقبل بيانات الإدخال غير الصالحة
- أن البرنامج ينتج دائمًا بيانات الإخراج المناسبة.

فيما يلي الأنواع الشائعة من الاختبار الإتلافي:

اختبار الأمان

يحاول "المتسللون" الخارجيون تعطيل البرنامج وإيقافه.

اختبار القبول

يحاول المستخدمون العاديون إنتاج مخرجات غير صحيحة ضمن مدخلات غير صحيحة.

اختبار الصندوق الأبيض

يتمتع المختبرون برؤية التعليمات البرمجية لتحديد الثغرات والعيوب المحتملة داخل البرنامج.

ويكشف الاختبار الإتلافي عن مشكلات غير موثقة وغير معروفة سابقًا تسمح للمبرمجين بحل خطة اختبار التراجع والتغذية الراجعة عليها، مما يقلل من احتمالية تكرارها في عمليات التطوير المستقبلية.

إعداد تقارير الاختبار

تتضمن تقارير الاختبار ملخصًا للاختبار - النجاح/الفشل/التخطي، وخطوات إعادة الإنتاج خاصة بحالات الفشل، وقوائم الأخطاء التي ظهرت.

ومن ثم، يكون الناتج الرئيسي للاختبار، بغض النظر عن النوع، هو Test Report "تقرير الاختبار". يلخص هذا التقرير الاختبار الذي تم إجراؤه ويعطي حالة النجاح أو الفشل أو التخطي لكل حالة اختبار يتم تنفيذها.

وبالنسبة للاختبارات التي تم اجتيازها، تكون حالة "النجاح" والتعليق على النتيجة الفعلية كافية للتقرير.

وبالنسبة للاختبارات الفاشلة، يلزم وجود حالة Fail "فشل" والتعليق على النتيجة الفعلية وsteps to reproduce "خطوات إعادة الإنتاج" الإضافية لدعم وتسهيل حل المسألة. ومن مصلحة المختبرين والمطورين مشاركة أكبر قدر ممكن من المعلومات حول الاختبار الفاشل لضمان اجتيازه في الاختبار التالي.

وبالنسبة للاختبارات التي تم تخطيها، فإن حالة Skipped "تم تخطيها" والتعليق على سبب تخطيها هي مفتاح التقرير. تشير الاختبارات التي تم تخطيها عادةً إلى أن موضوع الاختبار ليس جاهزًا للاختبار أو أن بيانات الإدخال أو الاختبار ليست كافية لإجراء الاختبار. فلا يمكن إجراء الاختبار بالكامل بدون بيانات الإدخال المناسبة. ولا يتم تصنيفه على أنه "فاشل"، بل هو أكثر من مجرد "محاولة مرة أخرى لاحقًا بمزيد من التفاصيل".

المهارات

المهارات المعرفية: العمليات والاستراتيجيات المعرفية:

- حل المسائل

الإبداع:

- الابتكار

المهارات الشخصية: أخلاقيات العمل/الضمير:

- المبادرة
- التوجيه الذاتي

مهارات التواصل الشخصي: العمل الجماعي والتعاون:

- التعاون
- العمل الجماعي

القيادة:

- عرض تقديمي فعال
- العرض الذاتي

ممارسة التقييم هدف التعلم (ج)

هدف التعلم (ج): تطوير برامج لتلبية متطلبات المستخدم

راجع تصميماتك من ممارسة التقييم في هدف التعلم (ب). واكتب تعليمات البرمجة لتطوير برنامج يلبي متطلبات المستخدم التي تم جمعها في تصميماتك.

(أ) ابدأ مشروع التطوير الخاص بك عن طريق تقسيم تصميماتك إلى مهام تطوير دقيقة.

(ب) قم بتقدير الوقت المطلوب لكل مهمة

(ج) قم بإنشاء خارطة طريق لأي مهام تطوير مستقبلية لبرنامجك

(د) اختر لغة البرمجة والنظام الأساسي لبرنامجك، مع ذكر أسباب الاختيارات

(هـ) قم بتطوير البرنامج وفقًا لتصميماتك، مع التأكد من أن التعليمات البرمجية الخاصة بك نظيفة ومنظمة جيدًا، مع ضمان تحديث كل مهمة مكتملة بالوقت الفعلي المستغرق.

(و) قم بإنشاء دليل مستخدم لمرافقة برنامجك

(ز) اختبر برنامجك باستخدام طرق الاختبار الوظيفية وإمكانية الاستخدام والتراجع والاختبار الإتلافي

يجب أن يكون الدليل الخاص بك لممارسة التقييم هذه في شكل برنامج عمل مع تعليمات حول كيفية إنشاء التطبيق وتشغيله، بما في ذلك أي متطلبات للنظام، مثل النظام الأساسي أو حزم تطوير البرامج الإضافية. تقرير ملاحظة من أحد الزملاء الذين يراقبون عملك، والتعليمات البرمجية المشروحة، ومجموعة كاملة من خطط الاختبار وبرامج النصوص والمقارنات بين الوقت الفعلي والوقت المتوقع الذي تستغرقه مهام التطوير.

أنشطة التقييم: أهداف التعلم (أ) و(ب) و(ج)

نشاط التقييم	هدف التعلم (أ)

مقدمة

لقد تم تعيينك كمبرمج متدرب في شركة هندسية صغيرة متخصصة في الطاقة الكهرومائية في الأردن. لقد كنت تبحث وتتعرف على بنيات البرمجة منذ أن بدأت العمل. لقد اقترح مديرك مشروعًا كبيرًا لتحسين كفاءة وأمان طريقة تخزين بيانات العملاء واستخدامها داخل الشركة. تقوم الشركة حاليًا بتخزين سجلات الاتصال الخاصة بالعملاء على جدول بيانات على شبكة الشركة. إنهم قلقون بشأن قابلية التوسع والأمان والكفاءة لتلك البيانات.

النشاط

يريدون منك إنشاء مستندات التصميم، بما في ذلك خطة الاختبار، لبرنامج يلبي هذه المتطلبات:

- برنامج يمكن تثبيته كتطبيق سطح مكتب (باستخدام WinForms أو WPF) على جميع محطات العمل داخل الشركة ويتطلب إدخال اسم المستخدم وكلمة المرور للمستخدم. يمكن التحقق من ذلك في ضوء قاعدة بيانات المستخدم على خادم ملفات الشركة أو الاستفادة من الدليل النشط للشركة.
- يجب أن يسمح البرنامج للمستخدم بالبحث عن عميل حسب اسم الشركة أو لقب جهة الاتصال.
- يجب أن يسمح البرنامج للمستخدم بإدخال معلومات عميل جديد وأن يكون قادرًا على تحرير المعلومات الموجودة. كما يجب أن تكون المعلومات الأساسية التي يجب أن يجمعها البرنامج هي اسم الشركة والعنوان ورقم الهاتف والبريد الإلكتروني وموقع الويب الخاص بالشركة واسم جهات الاتصال الأساسية ورقم الهاتف وعنوان البريد الإلكتروني الخاص بجهات الاتصال الأساسية.
- يجب أن يقوم البرنامج بتخزين بياناته في قاعدة بيانات SQLite آمنة على خادم ملفات الشركة.
- يجب عدم إتاحة إمكانية الوصول إلى المعلومات ما لم يتم تسجيل دخول المستخدم وتسجيل جميع الإجراءات لأغراض التدقيق.
- يجب أن يكون مسؤولو البرنامج قادرين على إعداد مستخدمين جدد داخل البرنامج.
- إذا طلب العميل إزالة بياناته، يجب أن تكون هناك طريقة سهلة لإزالة معلوماته نهائيًا.

معايير التقييم	هدف التعلم (أ)

النجاح	التفوق	الامتياز
هدف التعلم (أ): دراسة الأدوات والتقنيات المستخدمة في تطوير برنامج		
A.P1 وصف الأدوات المستخدمة في تطوير برنامج. **A.P2** وصف التقنيات المستخدمة في تطوير برنامج.	**A.M1** شرح الغرض من الأدوات والتقنيات المستخدمة في تطوير برنامج.	**A.D1** تقييم الغرض وأهمية الأدوات والتقنيات المستخدمة في تطوير برنامج.

نقطة مراجعة

كيف يمكن أن يفتقر البرنامج إلى الأمان، وما هي تقنيات الأمان التي يمكن إضافتها إلى البرنامج لجعله أكثر أمانًا؟ (فكر في ميزات الأمان التي تستخدمها في البرامج أو مواقع الويب الأخرى).

ما هي المنصات الأخرى التي يمكن تطوير البرنامج من أجلها؟

نشاط التقييم أهداف التعلم (ب) و(ج)

مقدمة

بعد المراجعة الدقيقة، وافق مديرك على مستندات التصميم الخاصة بك. ويود مديرك أن تخطط لتطوير البرنامج في إصدار ثلاثي المراحل. يجب أن يحتوي الإصدار الأول على المتطلبات التالية من الملخص الأصلي، مع باقي المتطلبات التي يجب اتباعها بالترتيب المفضل لديك.

فيما يلي المتطلبات الأساسية:

1. برنامج يمكن تثبيته على جميع محطات العمل داخل الشركة يتطلب إدخال اسم المستخدم وكلمة المرور للمستخدم. يمكن التحقق من ذلك في ضوء قاعدة بيانات المستخدم على خادم ملفات الشركة أو الاستفادة من الدليل النشط للشركة.
2. يجب أن يسمح البرنامج للمستخدم بالبحث عن عميل حسب اسم الشركة أو لقب جهة الاتصال.
3. يجب أن يسمح البرنامج للمستخدم بإدخال معلومات عميل جديد وأن يكون قادرًا على تحرير المعلومات الموجودة. كما يجب أن تكون المعلومات الأساسية التي يجب أن يجمعها البرنامج هي اسم الشركة والعنوان ورقم الهاتف والبريد الإلكتروني وموقع الويب الخاص بالشركة واسم جهات الاتصال الأساسية ورقم الهاتف وعنوان البريد الإلكتروني الخاص بجهات الاتصال الأساسية.
4. يجب أن يقوم البرنامج بتخزين بياناته في قاعدة بيانات SQLite آمنة على خادم ملفات الشركة.
5. يجب عدم إتاحة إمكانية الوصول إلى المعلومات ما لم يتم تسجيل دخول المستخدم وتسجيل جميع الإجراءات لأغراض التدقيق.

سيتعين عليك القيام بما يلي لتحقيق النجاح:

- تحديد مهام التطوير للمتطلبات الأساسية من خلال إنشاء قوائم أو بطاقات للعمل المطلوب، بما في ذلك تعريف المهام مثل:
  - إنشاء واجهات المستخدم
  - إنشاء أحداث مرتبطة بتفاعلات الواجهة
  - إنشاء طرق للإجراءات المطلوبة
  - ربط الطرق بالأحداث
  - تقدير وقت المطور، بالساعات، لكل مهمة مكلف بها المطور.
- توفير خارطة طريق للإصدار الأولي من البرنامج، والتاريخ المقدر للتسليم فيما يتعلق بباقي المتطلبات.

تابع أهداف التعلم (ب) و(ج)

- اختيار وإعطاء أسباب لغة البرمجة المطلوبة ومنصة التطبيق.
- تطوير التطبيق والتعليق على التعليمات البرمجية الخاصة بك لإظهار استخدام الأدوات والتقنيات التالية التي تغطيها هذه الوحدة.
- اختبار التطبيق وإصلاح أي مشكلات تم العثور عليها من الاختبار وإعادة الاختبار حتى لا تبقى أية مشكلات.
- إنشاء دليل مستخدم لدعم مستخدمي التطبيق الخاص بك باستخدام لقطات الشاشة وأدلة النص أو الفيديو.

يجب أن تتكون محفظة عملك من التصاميم وخطط الاختبار والنتائج والنسخ الإلكترونية من برنامجك. يجب أن يكون لديك دليل على تطبيقك قيد التشغيل في شكل دليل مستخدم.

معايير التقييم أهداف التعلم (ب) و(ج)

النجاح	التفوق	الامتياز
هدف التعلم (ب): تصميم برنامج لتلبية متطلبات المستخدم		
B.P3 تصميم برنامج لتلبية متطلبات المستخدم. **B.P4** ابتكار برامج نصوص للاختبار للتأكد من أن البرنامج يستوفي متطلبات المستخدم.	**B.M2** تصميم برنامج باستخدام واجهة مستخدم فعالة. **B.M3** ابتكار مجموعة من الاختبارات لتحديد المخاطر المحتملة، للتأكد من أن البرنامج يستوفي متطلبات المستخدم ووظائفه.	**BC.D2** تصميم وتخطيط وتطوير واختبار برنامج باستخدام مجموعة من أدوات وتقنيات البرامج والتأكد من أنه يلبي متطلبات المستخدم ويعمل على النحو المنشود دون أي أخطاء.
هدف التعلم (ج): تطوير برامج لتلبية متطلبات المستخدم		
C.P5 تخطيط مهام التطوير لبرمجة البرامج. **C.P6** تطوير برمجة البرامج باستخدام تقنيات البرمجة وأدوات التطوير.	**C.M4** تخطيط برمجة البرامج بواجهة مستخدم فعالة وتطويرها، باستخدام التصاميم والتقنيات والأدوات المختلفة لتمكين اختبار متطلبات المستخدم والتأكد من أنها تعمل جيدًا.	**BC.D2** تصميم وتخطيط وتطوير واختبار برنامج باستخدام مجموعة من أدوات وتقنيات البرامج والتأكد من أنه يلبي متطلبات المستخدم ويعمل على النحو المنشود دون أي أخطاء.

نقطة مراجعة

إذا اكتشف المستخدمون الأخطاء بعد صدور الإصدار الأول، فكيف يمكن أن يؤثر ذلك على خارطة الطريق؟

كيف تحدد أولويات المهام بعد الإصدار الأول؟

نصائح

سيساعدك بدء دليل المستخدم الخاص بك أثناء التطوير على تصميم البرنامج من منظور المستخدم. وإذا كان من الصعب توضيح كيفية استخدام البرنامج في الدليل، فربما تكون واجهتك معقدة للغاية؟ فكّر في تبسيط التصميم أو تقليل عدد النقرات لتنفيذ إجراء قياسي أو إضافة مربعات حوار مساعدة إلى واجهة المستخدم للمساعدة.

استكشف المزيد

أعد تطوير التطبيق المقرر استضافته على الويب. يمكن رفع معظم التعليمات البرمجية الوظيفية الخاصة بك إلى مشروع جديد يستهدف الويب. وأضف صفحة تسجيل دخول بمصادقة "ثنائية العوامل" إلى موقع الويب لتعزيز الأمان.

مسرد المصطلحات

C#: لغة في إطار مايكروسوفت للغات البرمجة. وهي تستند إلى لغة C، وهي لغة إجرائية، مع ميزات موجهة نحو الكائنات.

إنترنت الأشياء: "IoT"، عبارة عن تقنية تسمح لنا بإضافة جهاز إلى كائن خامل.

الإجراء: فعل أو طريقة يمكن أن يقوم بها شيء ما. تحدد عبارة "عمر يركب دراجته" الفعل الذي يمكن أن يؤديه عمر.

البت: رقم ثنائي (يمثله 0 أو 1) وأصغر وحدة بيانات يمكن للكمبيوتر معالجتها وتخزينها. يتكون البايت من 8 بت.

الحلقة اللانهائية: جزء من التعليمات البرمجية لا يحتوي على شرط خروج، وبالتالي يتكرر إلى الأبد.

الربع: فترة زمنية ضمن السنة التشغيلية للمؤسسة باستخدام تعريف "أرباع السنة"، على سبيل المثال، يغطي الربع الأول أو Q1 أشهر يناير وفبراير ومارس. ستعطي "نهاية الربع الثاني" وقتًا تقديريًا للتسليم حتى وقت ما خلال شهر يونيو.

الزيادة: تُعرف زيادة الرقم بمقدار واحد بالزيادة. ويمكن كتابة هذا في صورة a = a + 1 or a++. يُعرف ++ بعد المتغير باسم الزيادة اللاحقة.

الفئة: تعريف قالب للطرق والمتغيرات في نوع معين من الكائنات.

القيمة المنطقية: قيمة صواب أو خطأ، يمكن اختصارها إلى "bool".

الكتلة: أسطر التعليمات البرمجية التي تم تجميعها معًا.

المثيل: نسخة من كائن يمكن أن يكون له قيمه الخاصة في المعلمات ولكن لا يزال يعتمد على الكائن الأصلي الخاص به.

المعلمة: سمة أو خاصية أو وصف يعطي تعريفًا لشيء ما. تصف عبارة "عمر ذو شعر بني وعيون خضراء" خاصيتين لعمر.

النقصان: يُعرف تقليل الرقم بمقدار واحد بالنقصان. ويمكن كتابة هذا في صورة a = a – 1 or a--. يُعرف -- بعد المتغير باسم النقصان اللاحق.

النموذج: طريقة لتصنيف الكائنات بناءً على الميزات المشتركة.

باقي القسمة: ينتج قيمة متبقية من قسمة رقمين.

خارطة الطريق: جدول تخطيط رشيق لدعم الأهداف طويلة الأجل والمستقبلية للمشروع.

زائف: شيء مزيف يحاول أن يمر بوصفه الشيء الحقيقي.

عدد صحيح: العدد الصحيح هو رقم صحيح في البرمجة. ولا يمكن أن يحتوي على كسر أو رقم عشري.

قابل للتغيير: شيء قابل للتغيير.

نوع البيانات: سمة تصف جزءًا من البيانات وتخبر جهاز الكمبيوتر بكيفية قراءتها وكتابتها.